TIME SERIES FORECASTING USING TRANSFORMER ARCHITECTURE

ABSTRACT

Time Series Forecasting (TSF) plays a critical role in strategic decision-making across domains such as energy, transportation, and finance. While Transformer-based models have established a new standard for modeling long-range dependencies, they suffer from a fundamental limitation: the reliance on a fixed temporal resolution. This constraint often hinders the ability to simultaneously capture short-term high-frequency fluctuations and long-term global trends, leading to suboptimal performance on complex real-world data.

To overcome this challenge, this project proposes the Adaptive Multi-Resolution Transformer (AMR-Transformer). This novel architecture enables the model to dynamically learn and fuse features across varying temporal granularities. The system is constructed upon four core components: the Adaptive Resolution Module (ARM), Multi-Branch Processing, Resolution-Aware Attention, and Cross-Resolution Fusion. By integrating a learnable resampling mechanism with advanced patching techniques, the model effectively balances computational efficiency with deep contextual learning. Experimental evaluations on diverse benchmarks demonstrate that the AMR-Transformer successfully generalizes across different data characteristics, offering a robust and interpretable solution for time series forecasting without relying on rigid, pre-defined scales.

TABLE OF CONTENT

LIST OF FIGURES ..... **vi**

LIST OF TABLES ..... **viii**

ABBREVIATIONS ..... **ix**

CHAPTER 1. INTRODUCTION **1**

1.1 Problem statement 1

1.2 Issues with current models 1

1.3 Project objectives 2

1.4 Project Scope 3

CHAPTER 2. THEORETICAL BACKGROUND **5**

2.1 Time series forecasting problem 5

2.1.1 Formal definition *5*

2.1.2 Approaches *5*

2.2 Transformer architecture 6

2.2.1 Self-Attention mechanism *7*

2.2.2 Multi-Head Attention *9*

2.2.3 Positional encoding *10*

2.2.4 Complete encoder block *10*

2.3 Transformer for time series (TSF) 11

2.3.1 Challenges *11*

2.3.2 Related works *12*

2.4 Summary and research gap 14

CHAPTER 3. AMR-TRANSFORMER METHOD **15**

3.1 Architecture overview 15

3.2 Adaptive resolution module (ARM) 17

3.2.1 Learnable resampling 18

3.2.2 Adaptive router 19

3.3 Multi-Branch resolution processing 20

3.3.1 Local feature extraction 20

3.3.2 Continuous scale-aware positional encoding 20

3.4 Resolution-Aware attention 21

3.4.1 Unshared projections and alignment 22

3.4.2 Adaptive QKV fusion 22

3.4.3 Single global attention 23

3.5 Feed-Forward network and residual connections 24

3.5.1 Position-wise feed-forward network (FFN) 24

3.5.2 Residual connections and layer normalization 24

3.6 Prediction head and output generation 25

3.6.1 Flattening and linear projection 25

3.6.2 Denormalization (RevIN) 26

CHAPTER 4. EXPERIMENTAL SETUP 28

4.1 Datasets survey and description 28

4.1.1 Weather dataset 28

4.1.2 Traffic dataset 28

4.1.3 Electricity dataset 29

4.1.4 Exchange- rate dataset 29

4.1.5 Univariate Setting and Preprocessing *30*

4.2 Baseline Models 31

4.2.1 LSTM (Long Short-Term Memory) *32*

4.2.2 Transformer (Vanilla) *32*

4.2.3 DLinear (LTSF-Linear) *32*

4.2.4 PatchTST (Patch Time Series Transformer) *33*

4.3 Evaluation Metrics 33

4.3.1 Forecasting accuracy metrics *33*

4.3.2 Multi-horizon evaluation *34*

4.3.3 Computational efficiency metrics *35*

4.4 Implementation details 35

4.4.1 AMR-Transformer configuration *35*

4.4.2 Baseline models implementation details *37*

CHAPTER 5. RESULTS 40

5.1 Overall forecasting performance 40

5.2 Multi-horizon evaluation 44

5.2.1 Results on periodic datasets (Electricity & Traffic) *44*

5.2.2 Results on smooth & chaotic datasets (Weather & Exchange) *46*

5.3 Analysis of adaptive mechanisms 47

5.3.1 Electricity dataset: hierarchical structure and shift patterns *48*

5.3.2 Traffic dataset: modeling daily seasonality *49*

5.3.3 Exchange-rate dataset: adaptation to chaos *50*

5.3.4 Weather dataset: meteorological consistency *51*

5.3.5 Summary *52*

5.4 Computational efficiency 53

5.4.1 Model complexity and size *53*

5.4.2 Inference latency by dataset *54*

5.5 Qualitative analysis and visual comparison 56

5.5.1 Traffic dataset: capturing strong periodicity *56*

5.5.2 Electricity dataset: mitigating over-smoothing *57*

5.5.3 Weather dataset: balancing trend and stochasticity *59*

5.5.4 Exchange-rate dataset: trend following in chaos *60*

5.5.5 Summary *62*

CHAPTER 6. CONCLUSION 63

6.1 Summary of contributions 63

6.2 Key findings 63

6.3 Limitations 64

6.4 Future work 65

REFERENCES 67

LIST OF FIGURES

Figure 2.1: The transformer model architechture 7

Figure 2.2: Scaled Dot-Product Attention and Multi-Head Attention consists of several attention layers running in parallel. 9

Figure 3.1: Data flow diagram of the AMR Block 16

Figure 3.2: Data flow diagram of the AMR-Transformer architecture. 17

Figure 3.3: Data flow diagram of the Adaptive resolution module 18

Figure 3.4: Data flow diagram of the Learnable resampler 1D 19

Figure 3.5: Data flow diagram of the Continuous scale-aware positional encoding 21

Figure 3.6: Data flow diagram of the Advanced resolution-aware attention.....22

Figure 3.7: Data flow diagram of the RevIN 27

Figure 5.1: Performance comparison of forecasting models on the Electricity dataset (non-stationary). Lower values indicate better performance. 41

Figure 5.2: Performance comparison of forecasting models on the Traffic dataset (periodic). Lower values indicate better performance. 41

Figure 5.3: Performance comparison of forecasting models on the Weather dataset (smooth). Lower values indicate better performance.....42

Figure 5.4: Performance comparison of forecasting models on the Exchange-Rate dataset (chaotic). Lower values indicate better performance.....42

Figure 5.5: Visualization of alpha weights on electricity dataset across 3 layers.....48

Figure 5.6: Visualization of alpha weights on traffic dataset. 49

Figure 5.7: Visualization of alpha weights on exchange-rate dataset.....50

Figure 5.8: Visualization of alpha weights on weather dataset. 51

Figure 5.9: Visual comparison of forecasting results on traffic dataset. 56

Figure 5.10: Visual comparison of forecasting results on electricity dataset. 58

Figure 5.11: Visual comparison of forecasting results on weather dataset.....59

Figure 5.12: Visual comparison of forecasting results on exchange-rate dataset.....61

LIST OF TABLES

Table 4.1: Summary of experimental datasets 30

Table 4.2: AMR-Transformer model architecture 35

Table 4.3: Training protocol and optimization 36

Table 4.4: Summary of baseline configurations 39

Table 5.1: Multivariate time series forecasting results (univariate setting) prediction horizon: 15, look-back: 96. 40

Table 5.2: Multi-horizon performance on electricity datasets (kWh).....44

Table 5.3: Multi-horizon performance on traffic datasets (Occupancy).....45

Table 5.4: Multi-horizon performance on weather datasets (ºC).....46

Table 5.5: Multi-horizon performance on exchange-rate datasets (rate).....47

Table 5.6: Comparison of model complexity.....54

Table 5.7: Inference latency (ms/batch).....54

ABBREVIATIONS

AMR-Transformer	Adaptive Multi-Resolution Transformer
ARIMA	Autoregressive Integrated Moving Average
ARM	Adaptive Resolution Module
CNN	Convolutional Neural Network
ETS	Error, Trend, Seasonality
FFN	Feed-Forward Network
FFT	Fast Fourier Transform
FLOPs	Floating Point Operations
GELU	Gaussian Error Linear Unit
GRU	Gated Recurrent Unit
LSTM	Long Short-Term Memory
LSTF	Long Sequence Time Series Forecasting
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
MB	Megabytes
MHA	Multi-Head Attention
MLP	Multi-Layer Perceptron

MSE	Mean Squared Error
OT	Output Target (Observation Target)
PAM	Pyramidal Attention Module
PE	Positional Encoding
PeMS	Performance Measurement System (Caltrans)
QKV	Query, Key, Value
ReLU	Rectified Linear Unit
RevIN	Reversible Instance Normalization
RMSE	Root Mean Squared Error
RNN	Recurrent Neural Network
SMAPE	Symmetric Mean Absolute Percentage Error
SOTA	State-of-the-art
SVR	Support Vector Regression
TCN	Temporal Convolutional Network
TSF	Time Series Forecasting
UCI	University of California, Irvine (Machine Learning Repository)

CHAPTER 1. INTRODUCTION

1.1 Problem statement

Time Series Forecasting (TSF) is a core problem with a long history and extensive applications in many practical fields. From predicting stock prices in finance, forecasting power load demand in the energy sector, estimating urban traffic flow, to modeling hydro-meteorological phenomena, the ability to accurately predict future values based on historical data plays a key role in decision-making and strategic planning.

Traditional statistical methods such as ARIMA and ETS (Error, Trend, Seasonality) have shown effectiveness in modeling time series with relatively stable structures. However, the increasing complexity of real-world data, characterized by non-linearity, non-stationarity, and complex correlations across multiple time scales, has driven the rapid development of Deep Learning models.

Architectures such as Recurrent Neural Networks (RNN) and their variants like LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) became the standard due to their ability to capture sequential dependencies. In particular, the advent of the Transformer architecture (Vaswani, et al., 2017) revolutionized not only natural language processing but also the time series domain. The ability to model long-range dependencies efficiently through the self-attention mechanism has helped Transformer and its variants (such as Informer, Autoformer) achieve superior performance.

1.2 Issues with current models

Despite possessing many advantages, traditional Transformers and their variants still have an inherent limitation when applied to TSF problems: they process time series at a fixed resolution. This means the model views data at a single level of granularity across the entire input sequence.

Even effective improvement methods like PatchTST (Nie, Nguyen, Sinthong, & Kalagnanam, 2023), which divides the sequence into "patches" (sub-segments),

still use a fixed patch size, implying a fixed resolution imposed by humans. This limitation is suboptimal because a real-world time series often contains information components at various frequencies and scales. Recent studies on adaptive modeling, such as MultiResFormer, argue that relying on fixed patch lengths and strides imposes a rigid human-defined bias, leading to an inability to capture complex, evolving temporal dependencies in real-world data. Forecasting at different horizons requires the model to be sensitive to different types of information:

- Short-term patterns: To accurately forecast near time steps (e.g., 1-step forecasting), the model needs to capture local, sudden, and detailed fluctuations. This requires high resolution.
- Long-term patterns: To forecast distant time steps (e.g., 96-step forecasting), the model needs to identify global trends and seasonality. This requires low resolution to filter out noise and focus on macro structures.

"Forcing" the model to use a single resolution leads to a trade-off: if the resolution is too high, the model is overloaded with noise when trying to forecast long-term; if the resolution is too low, the model misses important details for short-term forecasting.

1.3 Project objectives

To address the above problem, this project proposes designing and developing a new Transformer architecture named Adaptive Multi-Resolution Transformer (AMR-Transformer).

The core idea of AMR-Transformer is to build an architecture capable of automatically learning and adapting temporal granularity instead of using a fixed resolution. The model is designed to automatically adjust the level of detail it focuses on at different processing layers, following a natural intuition:

- Early layers: Prioritize processing information at high resolution to capture fine-grained patterns and local details.
- Middle layers: Process information at medium resolution to identify intermediate patterns.

- Deep layers: Gradually shift to processing information at low resolution to focus on global trends and long-term seasonality.

Specific objectives of the project include:

1. Design and implement the complete AMR-Transformer architecture, including core components:
    - Adaptive Resolution Module (ARM): Module for generating multi-resolution representations.
    - Multi-Branch Resolution Processing & Resolution-Aware Attention: Parallel processing branches and separate attention mechanisms for each resolution.
    - Cross-Resolution Fusion: Mechanism for aggregating information between resolutions.
2. Train and evaluate the performance of the AMR-Transformer model on standardized benchmark datasets with diverse characteristics such as Weather, Traffic, and Electricity.
3. Analyze the model's "self-adaptive" capability (e.g., visualizing learned pooling or gating parameters) to verify the hypothesis regarding the use of different resolutions at different layers.
4. Compare the performance of AMR-Transformer with foundational models (traditional Transformer) and SOTA models (such as PatchTST) to evaluate the superiority of the proposed method.

1.4 Project Scope

This project focuses primarily on Univariate Time Series Forecasting. Limiting the scope to univariate problems aims to concentrate resources on designing, verifying, and analyzing the core component: the "adaptive multi-resolution" mechanism.

The main goal is to build a robust model and prove the effectiveness of the "adaptive multi-resolution" idea in improving forecasting accuracy across different

horizons. Extending the architecture to Multivariate TSF will be considered a future development direction after the univariate foundation has been successfully verified.

CHAPTER 2. THEORETICAL BACKGROUND

2.1 Time series forecasting problem

2.1.1 Formal definition

The univariate Time Series Forecasting (TSF) problem can be formulated as follows: Given a historical observation sequence X recorded at consecutive time steps:

$$X = \{x_1, x_2, \dots, x_T\} \in R^T \tag{2.1}$$

where $x_t \in R$ is the observed value at time t, and T is the length of the historical sequence (also called the "look-back window").

The goal is to predict τ future values of the time series:

$$\hat{Y} = \{\hat{x}_{T+1}, \hat{x}_{T+2}, \dots, \hat{x}_{T+\tau}\} \in R^{\tau} \tag{2.2}$$

where τ is called the forecast horizon.

This problem seeks a function (model) $f(\cdot)$ such that:

$$\hat{Y} = f(X) \tag{2.3}$$

This function $f(\cdot)$ is trained from historical data to minimize a loss function $\mathcal{L}$ (e.g., MSE, MAE) between the predicted value $\hat{Y}$ and the actual value Y in the future.

2.1.2 Approaches

2.1.2.1 Statistical methods

ARIMA (Autoregressive Integrated Moving Average): These models assume that future values can be linearly represented by past values (AR), past errors (MA), and use differencing (I) to handle non-stationarity. They are effective for series with relatively linear and stable structures.

ETS (Error, Trend, Seasonality): These models decompose the time series into three components: Error, Trend, and Seasonality, and then forecast each component individually.

2.1.2.2 Machine learning methods

Models such as Support Vector Regression (SVR), Random Forest, or Gradient Boosting (e.g., XGBoost) can be applied. They require a manual "feature engineering" step, where sliding windows are used to create (input, output) pairs for supervised learning.

2.1.2.3 Deep learning methods

Recurrent Neural Networks (RNN/LSTM/GRU): Architectures like LSTM (Liu, et al., 2022) and GRU are specifically designed to handle sequential data. They use "gates" to maintain a hidden state, allowing information from previous time steps to be stored and propagated, helping the model capture long-term dependencies.

Temporal Convolutional Networks (TCN) (Bai, Kolter, & Koltun, 2018): These use 1D Convolutions. The key point is that TCN uses dilated convolutions, allowing the model's "receptive field" to increase exponentially with the depth of the network. This enables TCN to look very far into the past without needing recurrent architectures, while also allowing for parallel processing.

2.2 Transformer architecture

The Transformer, introduced in the paper "Attention Is All You Need" (Vaswani, et al., 2017), redefined sequential processing models. Instead of using recurrence (RNN) or convolution (CNN), the Transformer relies entirely on the self-attention mechanism to model relationships between elements in a sequence.

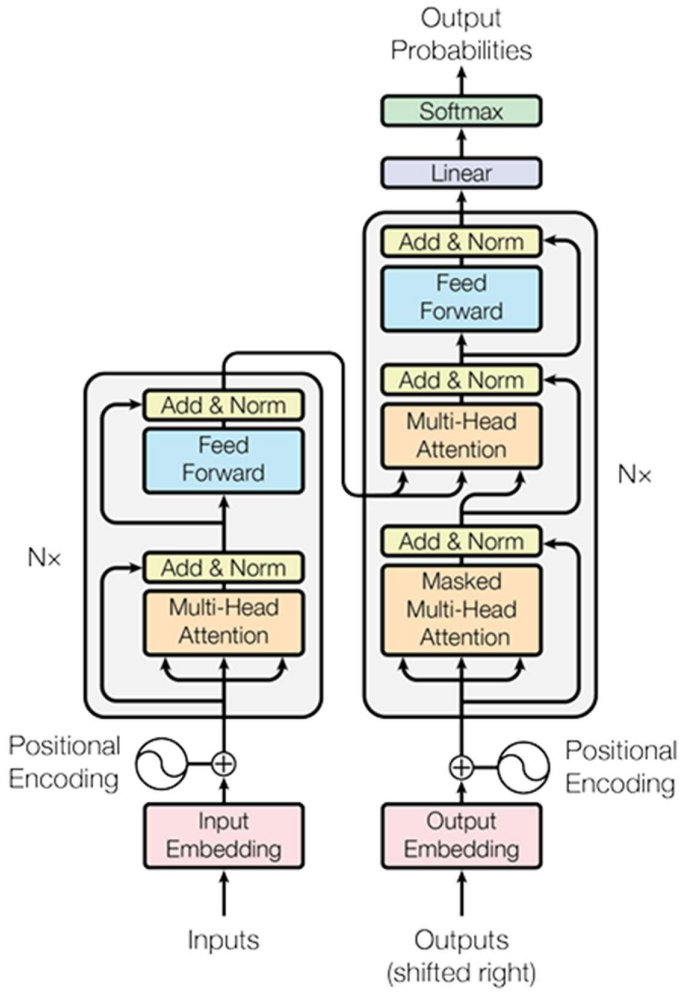

Figure 2.1: The transformer model architechture

2.2.1 Self-Attention mechanism

This is the heart of the Transformer. The idea is to compute a new representation for each token (here, each time step) by taking a weighted sum of all other tokens in the sequence. This weight (the level of "attention") is determined by the correlation between the tokens.

To do this, Self-Attention uses three learned vectors: Query (Q), Key (K), and Value (V).

To visualize this better, imagine you are in a library (the time series) to write a report on a topic (token x_t), then:

- Query (Q): Is the topic you are looking for (e.g., "short-term fluctuations"). This is the vector representing the current token, x_t, which is "asking" for information.
- Key (K): Is the label/keyword on the spine of each book (all tokens x_j). This is the vector representing token x_j, describing "what information do I contain?".
- Value (V): Is the actual content inside each book (all tokens x_j). This is the vector containing the actual information of token x_j.

The Attention process proceeds as follows:

1. Projection: From the input matrix X (e.g., $L \times d_{model}$), the model learns 3 weight matrices W^Q, W^K, W^V to generate Q, K, V:

$$Q = XW_Q \tag{2.4}$$

$$K = XW_K \tag{2.5}$$

$$V = XW_V \tag{2.6}$$

2. Calculate Scores: Take your Query (Q) and "compare" it with all Keys (K) in the library. This comparison is usually a dot product.

$$\text{Scores} = QK^T \tag{2.7}$$

The $Scores$ matrix has dimensions $L \times L$. The value $Scores_{ij}$ indicates how well the Query of token i matches the Key of token j.

3. Scaling: To stabilize gradients during training, the scores are divided by $\sqrt{d_k}$, where d_k is the dimension of the Key (and Query) vector.

$$\text{Scaled Scores} = \frac{QK^T}{\sqrt{d_k}} \tag{2.8}$$

If d_k is large, the product QK^T can take on very large values, pushing the softmax function (next step) into saturation regions (where gradients are zero), causing the model to fail to learn.

4. Calculate Weights: Apply the softmax function row-wise to the Scaled Scores matrix.

$$\text{Attention Weights} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \tag{2.9}$$

The result is an $L \times L$ matrix where each row sums to 1. This is the "attention weight" matrix. The value $Weights_{ij}$ is the "proportion" that token i should "attend" to information from token j.

5. Aggregation: Multiply the Attention Weights matrix by Value (V).

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.10}$$

The output Z (size $L \times d_v$) is the new representation of the sequence. Vector z_i (the i-th row of Z) is a weighted sum of all vectors V (i.e., $v_1, \ldots, v_L$), where the weights are determined by the correlation (calculated in step 4).

2.2.2 Multi-Head Attention

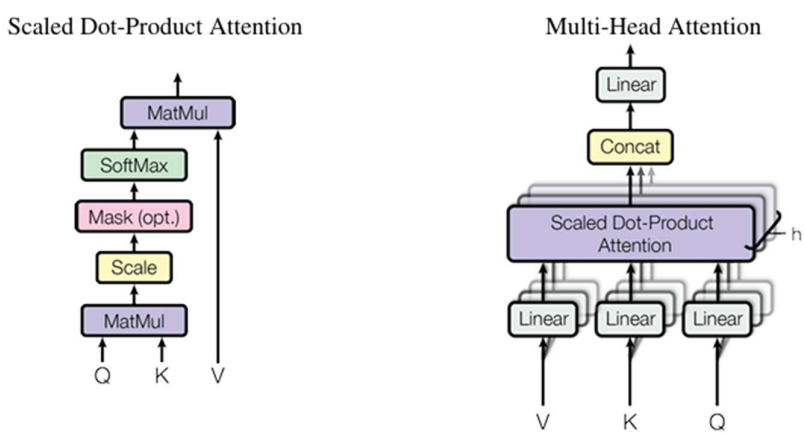

Figure 2.2: Scaled Dot-Product Attention and Multi-Head Attention consists of several attention layers running in parallel.

Instead of performing Attention just once, Multi-Head Attention performs h (e.g., $h = 8$) Attention operations in parallel. Each "head" i has separate weight matrices W_i^Q, W_i^K, W_i^V.

$$\text{head}_i = \text{Attention}\left(XW_Q^i, XW_K^i, XW_V^i\right) \tag{2.11}$$

Allows the model to "attend" to different types of information in different subspaces. For example, one "head" might focus on seasonality, while another focuses on sudden fluctuations.

The results from h "heads" are concatenated together, then passed through a linear layer W^O to aggregate and return to the original d_{model} dimension.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O \tag{2.12}$$

2.2.3 Positional encoding

Problem: The Self-Attention operation (calculating QK^T) is "permutation-invariant". If we swap the positions of x_2 and x_3, the output results are simply swapped accordingly. The model does not "know" the order of the sequence.Solution: Add a "Positional Encoding" (PE) vector to the input embedding vector.

$$X_{\text{final}} = X_{\text{embedding}} + PE \tag{2.13}$$

In the original paper (Vaswani, et al., 2017), PE is generated by sine and cosine functions at different frequencies, allowing the model to learn relative positions between tokens.

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{\text{model}}}) \tag{2.14}$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}}) \tag{2.15}$$

where pos is the position $(0, 1, \dots, L-1)$ and i is the index of the embedding dimension.

2.2.4 Complete encoder block

A Transformer Encoder block (the basic building block) consists of two main parts:

- Multi-head attention
- Position-wise Feed-Forward Network (FFN): A 2-layer feed-forward neural network (usually Linear -> ReLU -> Linear) applied to each position (token) independently.

Both of these parts use Residual Connections (He, Zhang, Ren, & Sun, 2016) and Layer Normalization (Ba, Kiros, & Hinton, 2016):

- Add & Norm 1:

$$Z' = LayerNorm\big(X + MultiHeadAttention(X)\big) \tag{2.16}$$

- Add & Norm 2:

$$Z_{out} = LayerNorm\big(Z' + FFN(Z')\big) \tag{2.17}$$

Residual connections ($X\ +\ ...$) help gradients flow back more easily through deep layers, avoiding the "vanishing gradient" phenomenon[16]. LayerNorm helps stabilize the training process.

2.3 Transformer for time series (TSF)

2.3.1 Challenges

A major issue with the self-attention mechanism lies in its computational and memory complexity of $\mathcal{O}(L^2)$, stemming from the calculation of the $L \times L$ matrix QK^T. When the sequence length L increases, for example reaching 5000 data points, this cost quickly becomes infeasible, consuming massive resources and processing time.

Besides, locality is also a significant limitation. Self-attention operates in a "global" manner, allowing every token to interact directly with every other token in the sequence. This can cause the model to overlook important local information, such as an upward or downward trend in just 10 consecutive data points – features that CNN-based models, such as Temporal Convolutional Network (TCN), capture very effectively.

Furthermore, non-stationarity in real-world time series further complicates the problem. Data distribution often changes over time, with phenomena such as increasing trends or fluctuating variance, making it difficult for the model to learn and predict stably. Therefore, standard normalization methods (like Z-score normalization on the entire dataset) are insufficient. Modern architectures often

require Instance Normalization or Reversible Normalization (RevIN) to adaptively handle these statistical changes per window.

2.3.2 Related works

2.3.2.1 Informer (2021) & Autoformer (2021)

The primary goal is to address the $\mathcal{O}(L^2)$ complexity in Long Sequence Time Series Forecasting (LSTF) problems, to enable efficient processing of long data sequences without resource bottlenecks.

Informer proposes the ProbSparse Attention mechanism, based on the observation that most Query-Key pairs in self-attention are not significant[27]. Instead of calculating everything, the model only selects and computes attention scores for the $\mathcal{O}(\log L)$ most "important" Keys for each Query, thereby significantly reducing complexity to $\mathcal{O}(L \log L)$.

Similarly, Autoformer completely replaces Self-Attention with an Auto-Correlation mechanism. This mechanism exploits the inherent periodicity of time series via Fast Fourier Transform (FFT) to identify similar dependencies between data points, also achieving $\mathcal{O}(L \log L)$ complexity.

Both Informer and Autoformer integrate decomposition blocks to separate trend and seasonal components in the time series, helping the model learn underlying features more effectively and improving long-term forecasting[31].

2.3.2.2 PatchTST (2023)

The main goal of this approach is to solve the $\mathcal{O}(L^2)$ complexity while improving locality in time series forecasting, opening up an effective new direction for long-term problems.

The core idea of PatchTST, a breakthrough work in the TSF field, lies in changing how data is viewed: instead of processing individual points (point-wise), the model divides the long time series L into non-overlapping patches (sub-segments), each with a fixed length P.

For example, with a sequence length $L = 1024$ and patch size $P = 16$, the model will divide it into $N = L/P = 64$ patches. Each patch consisting of P consecutive data points is flattened into a single vector, becoming an input token for the Transformer model.

Thanks to this, the Transformer now only needs to process a sequence of length $N = 64$, instead of $L = 1024$. Computational complexity drops sharply from $\mathcal{O}(L^2)$ to $\mathcal{O}(N^2)$, with $N \ll L$, making the model feasible on very long sequences. Furthermore, local information within each patch is naturally preserved, while the model learns to interact and predict based on relationships between patches – a harmonious combination of locality and global attention capabilities. However, a limitation of the original PatchTST is that it uses a fixed patch size (e.g., $P = 16$) for all layers. This implies a fixed resolution analysis. Our proposed AMR-Transformer extends this concept by enabling the model to process patches of different sizes simultaneously, thereby capturing multi-scale features more effectively.

2.3.2.3 Pyraformer (2022)

The main goal is to model time series at multiple resolutions (multi-resolution), an approach closely related to the AMR-Transformer project, to capture both detailed local information and long-term global trends.

The core idea revolves around building a Pyramidal Attention Module (PAM). The model uses convolutions to summarize nodes at a fine-grained level into nodes at a coarser-grained level, thereby forming a pyramid structure. In this structure, the top layers represent information at low resolution, reflecting long-term trends ; while the bottom layers retain high-resolution information, focusing on local details. The attention mechanism is then applied flexibly across different scales, allowing the model to effectively learn and combine features at multiple levels of abstraction.

2.3.2.4 MultiResFormer (2024)

The primary goal is to overcome the limitations of fixed-patching strategies (as seen in PatchTST) by introducing an adaptive multi-resolution mechanism,

enabling the model to capture temporal dependencies across various scales without manual hyperparameter tuning.

The core idea centers on dynamically determining the optimal patch lengths based on the intrinsic properties of the data. Unlike previous models that rely on a predefined grid or fixed hierarchy, MultiResFormer leverages Periodicity Detection (often utilizing Fast Fourier Transform - FFT) to identify the dominant cycles within the time series. Based on these detected periods, the model automatically assigns appropriate patch sizes to different processing branches.

This approach allows the architecture to process data at "natural" resolutions: capturing high-frequency noise with small patches and long-term seasonality with large patches. Furthermore, MultiResFormer typically employs a Shared Transformer Block across these branches, facilitating the learning of cross-scale interactions while maintaining parameter efficiency. This represents a significant step towards fully adaptive time series modeling, bridging the gap between patch-based efficiency and multi-scale feature extraction.

2.4 Summary and research gap

While models like Informer, PatchTST, and Pyraformer have made significant strides, and recent works like MultiResFormer have introduced multi-resolution concepts, a critical research gap remains.

Most existing multi-resolution approaches rely on either fixed hierarchical structures (Pyraformer) or pre-computed periodicity (MultiResFormer). They lack a fully "self-adaptive" mechanism that autonomously adjusts the importance (weights) of resolutions layer-by-layer during the learning process.

Therefore, this project focuses on developing the AMR-Transformer, which aims to fill this gap by allowing the model to learn how to prioritize resolutions dynamically across its depth (Layer-wise Adaptation), moving from local details in shallow layers to global trends in deep layers without rigid constraints.

CHAPTER 3. AMR-TRANSFORMER METHOD

Based on the core idea of enabling the model to self-adapt to different temporal resolutions, we propose the Adaptive Multi-Resolution Transformer (AMR-Transformer) architecture. This architecture is built upon four main components: the Adaptive Resolution Module (ARM), Multi-Branch Processing, Resolution-Aware Attention mechanisms, and Cross-Resolution Fusion.

3.1 Architecture overview

The overall architecture of the AMR-Transformer is founded on the principle of Integrated Multi-Resolution Processing. Departing from conventional Transformer models that process a single sequence at a fixed scale, this proposed model is designed to dynamically learn and fuse features from multiple temporal granularities within each processing layer. This approach allows the system to capture complex temporal dependencies more effectively than standard fixed-scale methods.

The data flow commences with an Input Normalization stage, where the input univariate sequence $X \in R^{T\times 1}$ is processed through Reversible Instance Normalization (RevIN). This step is critical for addressing non-stationarity and distribution shifts commonly found in time series data. Following normalization, the sequence is projected into a latent space of dimension D and augmented with Continuous Scale-Aware Positional Encoding. This encoding scheme effectively preserves temporal order information across different scales, preparing the data for the core processing blocks..

The core processing unit of the architecture consists of L identical AMR Transformer Blocks stacked sequentially. Within each individual block, the single input stream is transiently expanded into multiple resolution branches, specifically High, Mid, and Low, utilizing a Learnable Resampler. These branches are then independently processed by the Resolution-Aware Attention mechanism to capture features at varying granularities. Crucially, these multi-scale features are immediately

consolidated back into a single tensor via an Adaptive Weighted Fusion mechanism before being passed to the subsequent layer.

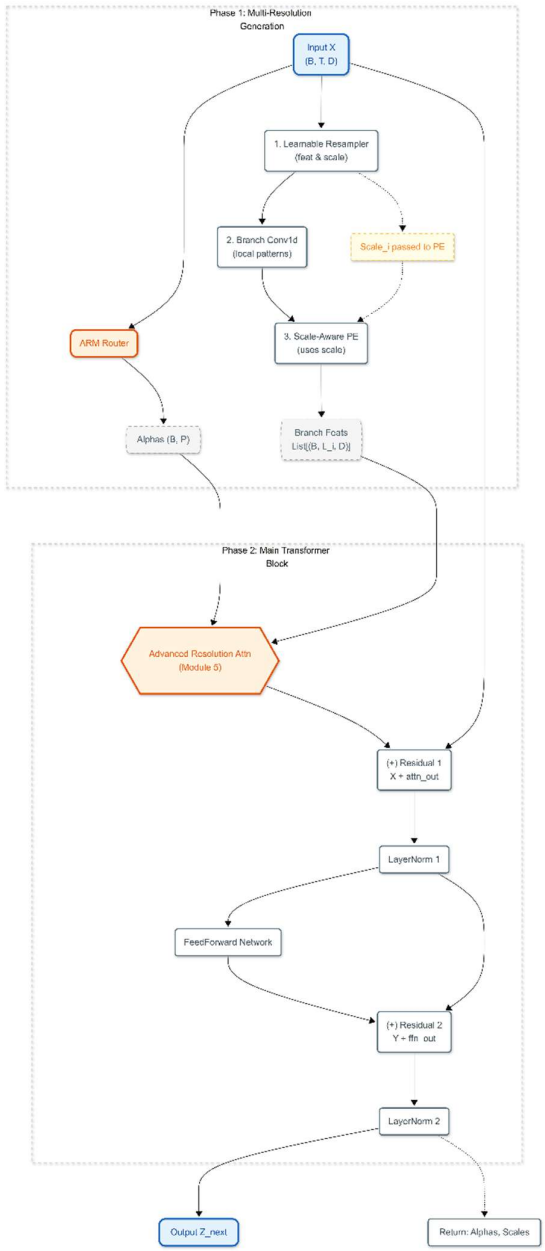

Figure 3.1: Data flow diagram of the AMR Block

In the final stage, the refined representation from the last Encoder block is flattened and projected by a Linear Head to generate the forecast sequence. To obtain the final output $\hat{Y}$, the sequence undergoes a denormalization process via RevIN,

reversing the initial normalization step. This ensures that the predicted values are returned to the original scale of the input data, completing the end-to-end forecasting pipeline.

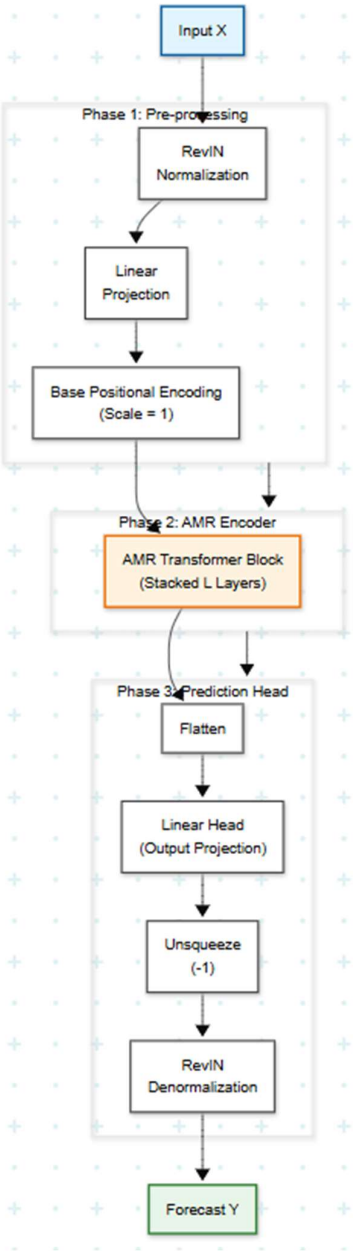

Figure 3.2: Data flow diagram of the AMR-Transformer architecture.

3.2 Adaptive resolution module (ARM)

This component represents the core innovation of the proposed architecture, designed to generate multi-resolution representations while simultaneously determining the relative importance of each resolution branch. In contrast to standard methodologies that rely on fixed pooling operations (such as Average Pooling with a stride of 2), the proposed ARM integrates a fully differentiable Learnable Resampling mechanism, allowing for more flexible and adaptive feature extraction.

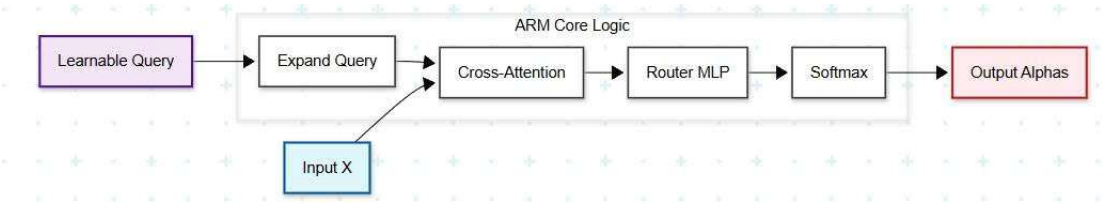

Figure 3.3: Data flow diagram of the Adaptive resolution module

3.2.1 Learnable resampling

Rather than employing rigid downsampling techniques, we introduce a continuous scaling mechanism to handle temporal variations. For every branch k, the model utilizes 1D Bilinear Interpolation to sample the input sequence into variable lengths based on a specific scale parameter. Because the computation of the sampling grid is fully differentiable, the network can backpropagate gradients directly to this scale parameter. This allows a branch effectively initialized with a period of 2 to fine-tune its scale to a value like 2.15 during training, enabling the model to better capture irregular seasonality that fixed sampling would miss.

$$X_k = \text{Resample}(X, \text{scale} = \sigma_k) \tag{3.1}$$

where**:**

- $X \in R^{T \times D}$: The input sequence.
- σ_k: The learnable scale parameter for branch k (resulting in new length $T_k = T/\sigma_k$).
- X_k: The output feature branch with a specific, learned temporal granularity.

Figure 3.4: Data flow diagram of the Learnable resampler 1D

3.2.2 Adaptive router

To dynamically quantify the reliability of each resolution branch, the ARM computes a set of attention weights, referred to as alphas. Instead of utilizing a simple linear layer, we employ a Query-Based Attention mechanism in which the input serves as both the Key and Value. A learnable Router Query vector attends to the input sequence to extract a global context vector, which is subsequently processed to produce the final mixing weights used in the Cross-Resolution Fusion step.

The global context vector is calculated as:

$$C = \text{Attention}(Q_{router}, K = X, V = X) \tag{3.2}$$

The mixing weights are then derived via:

$$\alpha = \text{Softmax}\big(\text{MLP}(C)\big) \tag{3.3}$$

where:

- Q_{router}: The learnable Router Query vector.
- C: The global context vector extracted from the input.
- $\alpha \in R^K$: The mixing weights (alphas) used to adaptively combine features from all branches.

3.3 Multi-Branch resolution processing

Following the generation of K branches with varying temporal lengths by the ARM, directly feeding these inputs into an Attention mechanism is suboptimal due to the alteration of temporal semantics caused by resampling. To mitigate this issue and ensure effective feature learning, we introduce two specialized processing stages, local feature extraction and scale-aware encoding, prior to the application of the Attention mechanism.

3.3.1 Local feature extraction

Before the model attempts to mix information globally, each individual branch is processed through a lightweight Convolutional layer. The primary objective of this step is to capture local dependencies effectively while simultaneously smoothing out any potential artifacts that may have been introduced during the resampling process. This layer employs a specific kernel configuration to help the model learn local patterns specifically tailored to the resolution scale of that particular branch.

$$X'_k = \mathrm{GELU}\big(\mathrm{Conv1d}(X_k)\big) \tag{3.4}$$

where:

- X_k: The input feature branch.
- $\mathrm{Conv1d}$: A 1D convolutional layer with a kernel size of 3.
- X'_k: The locally refined feature map.

3.3.2 Continuous scale-aware positional encoding

Standard Positional Encoding (PE) relies on fixed integer positions, which is insufficient for this architecture because the effective "time distance" between steps

varies according to the learned scale parameter. To address this discrepancy, we implement a Continuous Scale-Aware PE strategy. Instead of utilizing standard integer indices, the model calculates positions dynamically based on the learned scale. This mechanism is crucial for ensuring that the model preserves the correct notion of absolute time, even when sequence lengths are compressed, thereby facilitating precise temporal alignment during the fusion stage.

$$PE(pos, 2i) = \sin\left(\frac{pos \cdot \sigma}{10000^{2i/d_{model}}}\right) \tag{3.5}$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos \cdot \sigma}{10000^{2i/d_{model}}}\right) \tag{3.6}$$

where**:**

- pos: The original time step index.
- σ: The learned scale factor for the specific branch.
- d_{model}: The dimension of the feature embedding.
- $2i, 2i+1$: The indices for the even and odd dimensions of the embedding vector.

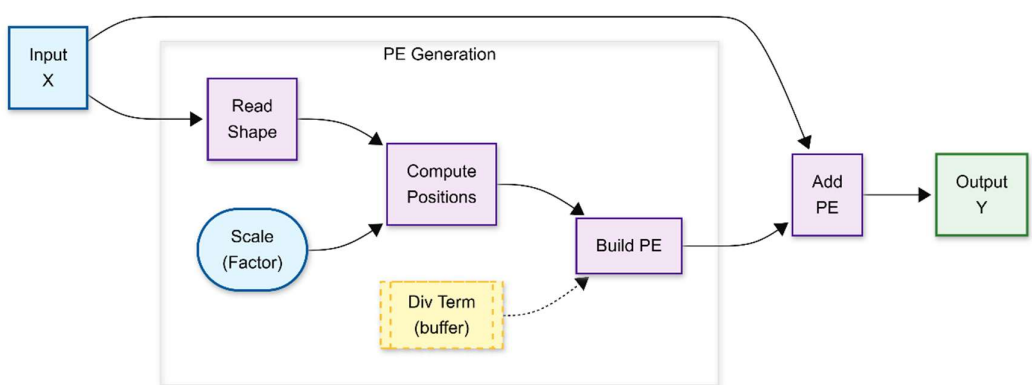

Figure 3.5: Data flow diagram of the Continuous scale-aware positional encoding

3.4 Resolution-Aware attention

To address the limitations of running independent Transformer blocks for each branch, which often fails to capture correlations between different scales, we implement a specialized Advanced Resolution-Aware Attention mechanism. This

component is designed to fuse information directly at the Query-Key-Value (QKV) level, ensuring a cohesive integration of multi-scale features before the attention operation is performed.

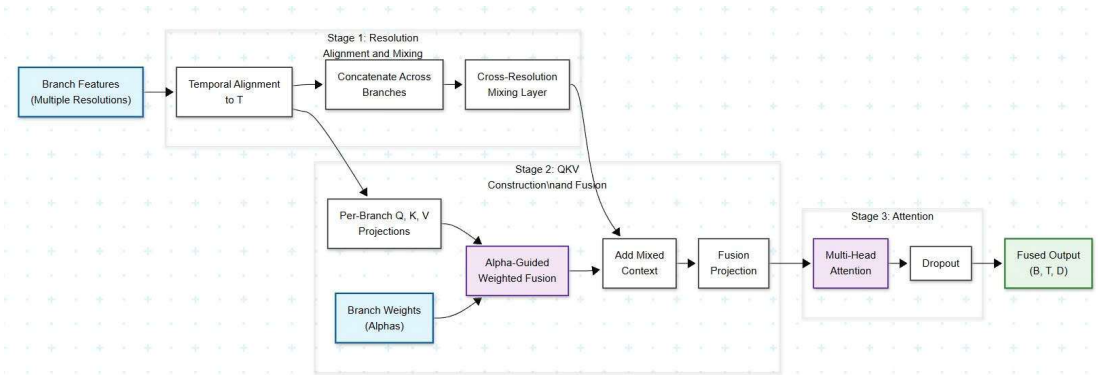

Figure 3.6: Data flow diagram of the Advanced resolution-aware attention

3.4.1 Unshared projections and alignment

The process begins by aligning features from the different resolution branches, $X_1, \dots, X_K$, to the original sequence length T via interpolation. Following this alignment, we employ Unshared Projections to capture resolution-specific patterns effectively. By maintaining separate Linear layers for each branch, the architecture allows specific branches to specialize; for instance, High-Resolution branches can focus on detailed fluctuations, while Low-Resolution branches capture global trends.

$$Q_k = W_Q^{(k)} X_k, \quad K_k = W_K^{(k)} X_k, \quad V_k = W_V^{(k)} X_k \tag{3.7}$$

where**:**

- X_k: The feature input from branch k.
- $W_Q^{(k)}, W_K^{(k)}, W_V^{(k)}$: The specific weight matrices for Query, Key, and Value projections for branch k.
- Q_k, K_k, V_k: The resulting Query, Key, and Value vectors for the specific resolution branch.

3.4.2 Adaptive QKV fusion

This stage represents the critical execution of Cross-Resolution Fusion within the Attention block. Utilizing the alpha weights generated by the Adaptive Resolution Module (ARM), the model fuses the distinct Query, Key, and Value vectors from all branches. Furthermore, a global mixing context is incorporated to ensure information flows across resolutions even prior to the attention mechanism.

$$Q_{fused} = \sum_{k=1}^{K} \alpha_k \cdot Q_k + C_{mix} \tag{3.8}$$

$$K_{fused} = \sum_{k=1}^{K} \alpha_k \cdot K_k + C_{mix} \tag{3.9}$$

$$V_{fused} = \sum_{k=1}^{K} \alpha_k \cdot V_k + C_{mix} \tag{3.10}$$

where**:**

- α_k: The adaptive weight for branch k provided by the ARM.
- C_{mix}: A global mixing context vector learned by concatenating all branches.
- $Q_{fused}, K_{fused}, V_{fused}$: The final fused vectors ready for the attention mechanism.

3.4.3 Single global attention

Finally, a standard Multi-Head Attention (MHA) mechanism is applied a single time to the fused vectors. This design choice is pivotal; by fusing at the QKV level, the model is forced to attend to a "hybrid" representation. This representation highlights the most relevant features from all resolutions simultaneously, governed by the learned Alphas, providing a significantly more effective analysis than processing branches in isolation.

$$Z = \text{Attention}\left(Q_{fused}, K_{fused}, V_{fused}\right) \tag{3.11}$$

where**:**

- Z: The output of the attention mechanism.
- $Q_{fused}, K_{fused}, V_{fused}$: The unified multi-resolution input vectors.

3.5 Feed-Forward network and residual connections

To complete the architecture of the AMR-Transformer Block, the unified feature representation obtained from the Resolution-Aware Attention mechanism undergoes non-linear processing and stabilization steps. This phase adheres to standard Transformer design principles but is specifically applied to the adaptively fused multi-resolution features, ensuring the refined signals are properly transformed before propagation.

3.5.1 Position-wise feed-forward network (FFN)

The output from the attention layer is passed to a Position-wise Feed-Forward Network. This sub-layer consists of two linear transformations interspersed with a non-linear activation function, which is applied identically and separately to each position in the sequence. In our implementation, we utilize the GELU (Gaussian Error Linear Unit) activation function, as it has been demonstrated to offer superior performance compared to ReLU in Transformer architectures. The dimensionality of the inner layer is expanded by a factor of 4 ($d_{ff} = 4 \times d_{model}$) to enable the model to capture more complex representations before being projected back to the original dimension.

$$FFN(x) = \text{Linear}_2\left(\text{Dropout}\left(\text{GELU}\left(\text{Linear}_1(x)\right)\right)\right) \tag{3.12}$$

where:

- x: The input vector from the attention layer.
- Linear_1: A linear layer projecting input from d_{model} to $4d_{model}$.
- Linear_2: A linear layer projecting input from $4d_{model}$ back to d_{model}.

3.5.2 Residual connections and layer normalization

To facilitate the effective training of deep networks and mitigate the vanishing gradient problem, we employ Residual Connections followed by Layer Normalization (Add & Norm) around each of the two sub-layers (Attention and FFN).

The processing flow within the final stage of the Encoder block involves two distinct normalization steps.

First, the Post-Attention Normalization adds the output of the Attention mechanism (Z_{attn}) to the block's input (Z_{in}) and normalizes the result:

$$Z' = \text{LayerNorm}\big(Z_{in} + \text{Dropout}(Z_{attn})\big) \tag{3.13}$$

Second, the Post-FFN Normalization adds the output of the FFN to the intermediate result (Z') and normalizes it again to produce the final block output:

$$Z_{out} = \text{LayerNorm}\Big(Z' + \text{Dropout}\big(FFN(Z')\big)\Big) \tag{3.14}$$

where:

- Z_{in}: The input to the current Transformer block.
- Z_{attn}: The output from the Resolution-Aware Attention mechanism.
- Z': The intermediate normalized tensor.
- Z_{out}: The final output of the block, serving as input for the next layer or the Prediction Head.

3.6 Prediction head and output generation

Upon the completion of processing through L layers of AMR-Transformer Blocks, the model yields a highly refined representation, denoted as $Z^{(L)}$. This tensor effectively encodes multi-resolution temporal dependencies extracted from the input. To translate this latent representation into a concrete forecast, the data undergoes a final transformation process within the prediction head, comprising three distinct stages: flattening, linear projection, and denormalization.

3.6.1 Flattening and linear projection

Given that the objective is to forecast a horizon of length τ, the temporal and feature dimensions of the final encoder output are first flattened. This operation converts the matrix into a comprehensive 1D feature vector, ensuring that all accumulated information across time steps and feature dimensions is preserved in a single contiguous array.

$$Z_{flat} = \text{Flatten}\left(Z^{(L)}\right) \in R^{T \cdot D} \tag{3.15}$$

Subsequently, a simple Linear layer is employed to project this high-dimensional vector directly into the target forecast horizon dimension.

$$\hat{Y}_{norm} = \text{Linear}\left(Z_{flat}\right) \in R^{\tau} \tag{3.16}$$

where:

- $Z^{(L)}$: The output tensor from the final AMR-Transformer Encoder layer ($T \times D$).
- Z_{flat}: The flattened feature vector ($T \cdot D$).
- τ: The length of the forecasting horizon.
- $\hat{Y}_{norm}$: The predicted forecast sequence (still in normalized scale).

3.6.2 Denormalization (RevIN)

As established in the initial data processing stage, the input data undergoes Reversible Instance Normalization (RevIN) to mitigate non-stationarity. Consequently, the output generated by the linear projection is currently in a normalized state. To derive meaningful forecast values that correspond to the original scale (e.g., real traffic volume), the inverse operation must be applied using the statistical properties, mean and standard deviation, stored during the input phase.

$$\hat{Y} = \hat{Y}_{norm} \times \sigma + \mu \tag{3.17}$$

where:

- μ: The mean of the input sequence (calculated and stored during input normalization).
- σ: The standard deviation of the input sequence.
- $\hat{Y}$: The final forecasted output in the original data scale.

The resulting $\hat{Y}$ constitutes the final output of the AMR-Transformer, which is subsequently compared against the ground truth to compute the loss during the training process.

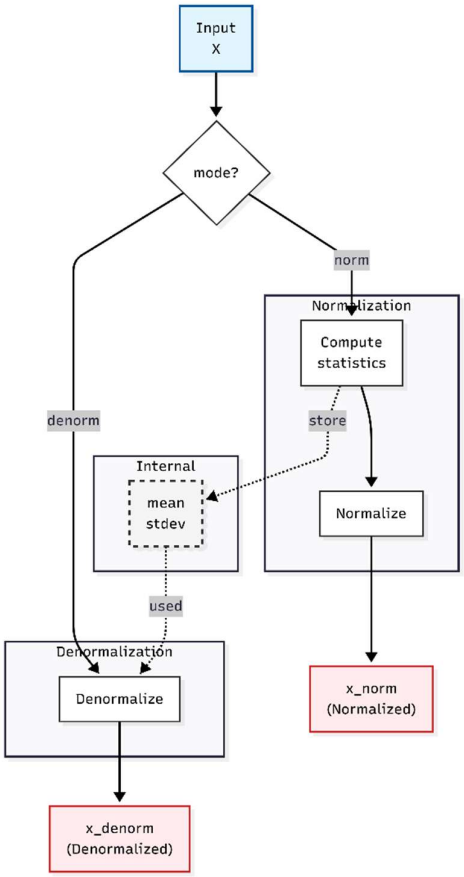

Figure 3.7: Data flow diagram of the RevIN

CHAPTER 4. EXPERIMENTAL SETUP

To comprehensively evaluate the effectiveness and adaptability of the proposed AMR-Transformer, we conducted extensive experiments on four widely used real-world benchmark datasets. These datasets cover diverse domains, Meteorology, Traffic, Energy, and Finance, each exhibiting unique temporal characteristics. We compared our model against several state of the art (SOTA) and classical baselines under a strict univariate setting.

4.1 Datasets survey and description

We utilized four standard datasets that are prevalent in Long-term Time Series Forecasting (LTSF) research. A summary of their statistics is provided in Table 4.1.

4.1.1 Weather dataset

- Source and origin: Collected by the Max Planck Institute for Biogeochemistry in Jena, Germany. This dataset was standardized as a benchmark in the Autoformer paper (Wu, Xu, Wang, & Long, 2021).
- Scale and attributes: The dataset contains 52,696 timesteps with 21 meteorological indicators, such as air temperature, pressure, humidity, and wind speed.
- Frequency: Recorded every 10 minutes.
- Characteristics: The data exhibits strong daily and yearly periodicity typical of natural phenomena. While the overall trends are smooth, it contains local fluctuations due to sudden weather changes.
- Experimental goal: To test the model's ability to capture continuous environmental variations and smooth periodic patterns.

4.1.2 Traffic dataset

- Source and origin: Sourced from the Caltrans Performance Measurement System (PeMS) by the California Department of Transportation. It was introduced in LSTNet (Lai, Chang, Yang, & Liu, 2018).

- Scale and attributes: It records the road occupancy rates (values between 0 and 1) from 862 sensors deployed on San Francisco Bay Area freeways. The dataset contains 17,544 timesteps.
- Frequency: Recorded hourly.
- Characteristics: This dataset demonstrates extremely strong periodicity. It shows clear daily patterns (morning/evening rush hours) and weekly patterns (significant differences between weekdays and weekends).
- Experimental goal: This is an ideal dataset to verify the "Multi-Resolution" hypothesis: whether the model can automatically learn short-term scales for daily peaks and long-term scales for weekly trends.

4.1.3 Electricity dataset

- Source and origin: Obtained from the UCI Machine Learning Repository and processed by (Lai, Chang, Yang, & Liu, 2018), covering the period from 2016 to 2019.
- Scale and attributes: Tracks the hourly electricity consumption (kWh) of 321 clients. The dataset comprises 26,304 timesteps.
- Frequency: Recorded hourly.
- Characteristics: Unlike Traffic, this dataset is highly non-stationary. It features numerous spikes and distribution shifts caused by changing user behaviors and seasonal demands.
- Experimental goal: To challenge the model's robustness against non-stationarity and irregular spikes.

4.1.4 Exchange- rate dataset

- Source and origin: A collection of daily exchange rates for eight countries (Australia, British, Canada, Swiss, China, Japan, New Zealand, Singapore) spanning from 1990 to 2016. Standardized by LSTNet (Lai, Chang, Yang, & Liu, 2018).

- Scale and attributes: Contains 7,588 timesteps for 8 currency features.
- Frequency: Recorded daily.
- Characteristics: This represents a chaotic system. Financial data lacks clear periodicity and is heavily influenced by macroeconomic factors, making it highly stochastic and difficult to forecast.
- Experimental goal: To evaluate the adaptability of the Learnable Resampling mechanism in a scenario where no obvious repetitive patterns exist.

Table 4.1: Summary of experimental datasets

Dataset	**Samples (Timesteps)**	**Features**	**Frequency**	**Key Characteristics**	**Split Ratio**
Weather	52.696	21	10 min	Smooth, Periodic, Continuous	7:1:2
Traffic	17.544	862	1 Hour	Strong, Periodicity, (Daily/Weekly)	7:1:2
Electricity	26.304	321	1 Hour	Non-stationary, Spikes, Shifts	7:1:2
Exchange	7.588	8	1 Day	Chaotic, Stochastic, No Periodicity	7:1:2

4.1.5 Univariate Setting and Preprocessing

Although the source datasets generally contain multivariate data, we implemented a specific preprocessing strategy to strictly adhere to the project scope of Univariate Forecasting. We adopt the "Channel Independence strategy" (Nie et al., 2023), which fundamentally alters how variable correlations are handled. Rather than modeling the inter-dependencies between different variables, such as using humidity to predict temperature, each channel within the multivariate dataset is treated as an isolated, independent univariate time series. In this configuration, the AMR-

Transformer utilizes a single shared set of weights across all these univariate sequences, allowing it to learn generalizable temporal patterns without relying on cross-variable correlations.

Regarding data management and preparation, specific protocols were established for consistency and stability. We designate the final column of the dataset (typically labeled as OT in Weather datasets or the last sensor/client in others) as the representative target variable for evaluation and visualization purposes. To prevent data leakage, all datasets are split strictly according to chronological order. Furthermore, prior to entering the model, the data undergoes instance normalization via Reversible Instance Normalization (RevIN). This step is crucial for mitigating distribution shifts (non-stationarity) and ensuring stable training dynamics.

Experimental configuration details:

- Data splitting strategy:

$$\text{Train:Validation:Test} = 7:1:2$$

The split is performed chronologically to respect the temporal order of the data.

- Target selection:

$$Y_{target} = X_{[:,-1]}$$

The last column is selected as the representative target for reporting metrics.

- Channel independence:

$$X_{multi} \in R^{T\times C} \xrightarrow{resha} X_{uni} \in R^{(T\cdot C)\times 1}$$

The model views the data as C independent variations of a single variable, sharing the same model parameters.

4.2 Baseline Models

To rigorously evaluate the performance of the AMR-Transformer, we benchmark it against four representative models. These baselines were carefully selected to cover the entire spectrum of forecasting methodologies, ranging from classical deep learning approaches (RNNs) to foundational architectures (Vanilla

Transformer) and recent State-of-the-Art (SOTA) methods, including both Linear-based and Patch-based Transformers.

4.2.1 LSTM (Long Short-Term Memory)

The Long Short-Term Memory (LSTM) network serves as our classical baseline for sequential data processing. As a variant of Recurrent Neural Networks (RNN), LSTM was specifically designed to address the vanishing gradient problem inherent in standard RNNs. It utilizes a sophisticated gating mechanism, comprising input, forget, and output gates, to regulate the flow of information, thereby enabling the capture of long-term sequential dependencies. Including LSTM in this benchmark is essential to demonstrate the architectural advantages of the AMR-Transformer's self-attention mechanism and parallel processing capabilities over the iterative, step-by-step processing nature of recurrent architectures.

Reference: (Hochreiter & Schmidhuber, 1997). Long short-term memory.

4.2.2 Transformer (Vanilla)

We also evaluate the original "Vanilla" Transformer architecture, the foundational model that revolutionized sequence modeling. Unlike RNNs, this architecture relies entirely on the self-attention mechanism to model global dependencies without the need for recurrence or convolutions. For our experiments, we utilize the canonical encoder structure operating with fixed-resolution attention. Since the AMR-Transformer is an evolutionary development of this architecture, this comparison is critical. It isolates and highlights the specific contributions of our Adaptive Multi-Resolution mechanism, contrasting it directly against the standard fixed-resolution attention approach.

Reference: (Vaswani, et al., 2017). Attention is all you need.

4.2.3 DLinear (LTSF-Linear)

To justify the computational complexity of our proposed model, we include DLinear as a baseline. DLinear challenges the prevailing trend of complex

Transformers by proposing a simple yet effective architecture based on a single-layer Linear network (MLP). It employs a decomposition scheme that separates the time series into Trend and Seasonal components, processing each via independent linear layers. As a strong, lightweight baseline that often outperforms complex models on datasets with clear patterns, DLinear serves as a necessary benchmark to ensure that the AMR-Transformer provides genuine performance gains over simple linear mappings.

Reference: (Zeng, Chen, Zhang, & Xu, 2022). Are transformers effective for time series forecasting?

4.2.4 PatchTST (Patch Time Series Transformer)

Finally, we benchmark against PatchTST, the current State-of-the-Art (SOTA) model in Long-term Time Series Forecasting. PatchTST introduced two pivotal concepts: Patching, which segments the time series into sub-sequences to preserve local semantics, and Channel Independence, where multivariate series are processed as independent univariate channels. As the direct competitor and foundational inspiration for our patching strategy, PatchTST is vital for this evaluation. While PatchTST relies on a fixed patch size (representing a single resolution), the AMR-Transformer extends this concept by introducing learnable, multiple resolutions. Outperforming PatchTST would provide strong empirical evidence for the efficacy of the Learnable Resampling and Adaptive Fusion mechanisms proposed in this thesis.

Reference: (Nie, Nguyen, Sinthong, & Kalagnanam, 2023). A time series is worth 64 words: Long-term forecasting with transformers.

4.3 Evaluation Metrics

4.3.1 Forecasting accuracy metrics

The performance of AMR-Transformer is comprehensively evaluated through 05 common error statistical metrics. Let $Y = \{y_1, \dots, y_n\}$ be the actual values and $\hat{Y} = \{\widehat{y_1}, \dots, \widehat{y_n}\}$ be the predicted values.

- Mean Squared Error (MSE): Measures the average squared difference between estimated values and the actual value. It is highly sensitive to large errors (outliers).

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- Root Mean Squared Error (RMSE): Represents the square root of MSE. It brings the error metric back to the same unit as the original data, making it easier to interpret the magnitude of the error.

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}}$$

- Mean Absolute Error (MAE): Calculates the average of absolute errors. It provides a linear score that effectively reflects the average model accuracy without heavily penalizing outliers like MSE.

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- Mean Absolute Percentage Error (MAPE): Measures the prediction accuracy as a percentage, which is useful for comparing performance across datasets with different scales.

$$\mathrm{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|$$

- Symmetric Mean Absolute Percentage Error (SMAPE): A variant of MAPE designed to address issues when actual values are near 0 and limits error within the 0-200% range.

$$\mathrm{SMAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\frac{|y_i - \hat{y}_i|}{(|y_i| + |\hat{y}_i|)/2}$$

4.3.2 Multi-horizon evaluation

Since the goal is Long-term Time Series Forecasting (LTSF), evaluating the average error over the entire sequence is insufficient. We further analyze the model's degradation over time by reporting metrics at specific future horizons:

- Step 1: Immediate forecast (Next step).
- Step 6: Short-term forecast.
- Step 10: Medium-term forecast.
- Step 15: Long-term forecast (End of horizon).

This granular analysis allows us to verify if the Multi-Resolution mechanism helps the model maintain accuracy for distant future steps (Low-Res branch contribution) compared to baselines.

4.3.3 Computational efficiency metrics

Beyond accuracy, practical deployment requires efficiency. We evaluate:

- Parameters (Params): The total number of trainable weights, indicating model size.
- FLOPs (Floating Point Operations): The number of operations required for a single forward pass, serving as a proxy for computational latency.
- Memory usage: The peak GPU memory consumption (MB) during inference, ensuring the model fits within constrained hardware resources.

4.4 Implementation details

4.4.1 AMR-Transformer configuration

The proposed model is configured with parameters balanced between computational efficiency and forecasting capability. The configuration is divided into structural specifications and training settings.

Table 4.2: AMR-Transformer model architecture

Category	**Parameter**	**Value**	**Variable Name / Note**
Input/Output	Look-back Window	96	INPUT_LEN
	Prediction Horizon	15	OUTPUT_LEN

Backbone	Model Dimension (dmodel)	64	COMMON_D_MODEL
	Attention Heads	4	COMMON_NHEAD
	Model Depth	3 Layers	NUM_LAYERS
	Dropout	0.2	COMMON_DROPOUT (Applied after Attention & FFN)
Normalization	Input Normalization	RevIN	Reversible Instance Normalization to handle non-stationarity.
Adaptive Module	Initial Periods	[1, 2, 4, 8, 12, 24]	universal_periods (Initial scales for ARM branches).
	Resampling Mechanism	Learnable	LearnableResampler1D (Scales are fine-tuned via gradient descent).

Table 4.3: Training protocol and optimization

Category	**Parameter**	**Value**	**Description**
Environment	Framework	PyTorch	Implemented on Google Colab (GPU-enabled).
	Data Split	Chronological	shuffle=False for Validation/Test sets.
Optimizer	Type	Adam	torch.optim.Adam
	Learning Rate	0.0005	LEARNING_RATE
	Batch Size	32	BATCH_SIZE
Loop Control	Max Epochs	40	NUM_EPOCHS
	Early Stopping	5 Epochs	Stops if Validation Loss does not improve for 5

			consecutive epochs (PATIENCE).
	Loss Function	MSE	Mean Squared Error (nn.MSELoss).

4.4.2 Baseline models implementation details

To ensure a rigorous and fair comparison, all baseline models were implemented with configurations that closely mirror the AMR-Transformer's environment (same Input/Output length, same RevIN normalization). Specifically, we standardized the hidden dimensions and dropout rates across models to ensure that differences in performance are attributable to architectural innovations rather than parameter size discrepancies.

4.4.2.1 LSTM (Long Short-Term Memory)

- Architecture: A standard stacked LSTM network designed for sequential processing.
- Input/Output: Data is projected linearly to the hidden dimension before entering LSTM cells.
- Configuration:
    - Hidden Layers: 2 (num_layers=2).
    - Hidden dimension: 64 (d_model=64), strictly matching the AMR model's latent size.
    - Dropout: 0.2 applied between the stacked LSTM layers.

4.4.2.2 Vanilla Transformer

- Architecture: Standard Transformer Encoder based on Vaswani et al. (2017).
- Mechanism: Uses standard Scaled Dot-Product Attention with Fixed Resolution (no resampling).
- Configuration:

- Depth: 3 Encoder Layers (num_layers=3), matching the depth of the AMR model.
- Attention: 4 Heads (nhead=4).
- Positional encoding: Standard Sinusoidal PE.
- Feed-forward network: Default PyTorch configuration (dim_feedforward=2048).
- Dropout: 0.2.

4.4.2.3 DLinear

- Architecture: A simple one-layer Linear model with a decomposition block (Trend + Seasonality).
- Mechanism:
  - Trend branch: Input passes through a Moving Average filter, then a Linear layer.
  - Seasonal branch: Residual (Input - Trend) passes through a separate Linear layer.
  - Final output: Sum of the two branches.
- Configuration:
  - Moving average kernel: 25 (kernel_size=25). Used to extract the Trend component.

4.4.2.4 PatchTST

- Architecture: Channel-independent Transformer with Patching mechanism.
- Mechanism: Segments the time series into overlapping patches to capture local semantics.
- Configuration:
  - Patch Length (P): 16 (patch_len=16).
  - Stride (S): 8 (stride=8). This creates a 50% overlap between patches to preserve information continuity.
  - Model Depth: 3 Layers (num_layers=3).

- Hidden Dimension: 64 (d_model=64). Note: While the original PatchTST paper often utilizes larger dimensions (e.g., 128), we explicitly restrict it to 64 in this experiment to maintain a fair parameter count comparison with our AMR-Transformer.
- Attention: 4 Heads (nhead=4).
- Dropout: 0.2.

Table 4.4: Summary of baseline configurations

Model	**Key Parameters**	**Specific Settings**
LSTM	Layers / Hidden / Dropout	Layers=2 Hidden=64 Dropout=0.2
Transformer	Layers / Heads / Dropout	Layers=3 Heads=4 FFN=2048 Dropout=0.2
DLinear	Kernel Size	Moving Average Kernel = 25
PatchTST	Patch / Stride / Dim	Patch=16 Stride=8 d_{model}=64 Dropout=0.2

CHAPTER 5. RESULTS

5.1 Overall forecasting performance

Table 5.1: Multivariate time series forecasting results (univariate setting) prediction horizon: 15, look-back: 96.

Dataset	**Model**	**MSE**	**MAE**	**RMSE**	**MAPE (%)**	**SMAPE (%)**
Electricity (Non-stationary)	AMR	52,344.66	163.45	228.79	4.77	4.71
	PatchTST	52,844.55	165.72	229.88	4.84	4.79
	LSTM	55,434.59	168.52	235.45	4.92	4.89
	Transformer	60,596.18	175.61	246.16	5.16	5.10
	DLinear	80,800.72	203.08	284.25	5.95	5.90
Traffic (Periodic)	AMR	0.000037	0.0036	0.0061	14.89	15.16
	PatchTST	0.000041	0.0039	0.0064	18.19	20.26
	Transformer	0.000042	0.0042	0.0065	19.06	21.27
	LSTM	0.000048	0.0041	0.0069	17.33	17.43
	DLinear	0.000065	0.0053	0.0081	21.86	20.16
Weather (Smooth)	DLinear	48.53	4.62	6.97	1.05	1.05
	AMR	63.94	5.72	8.00	1.29	1.29
	PatchTST	67.67	5.82	8.23	1.33	1.32
	Transformer	75.72	6.28	8.70	1.42	1.42
	LSTM	95.47	7.21	9.77	1.64	1.63
Exchange (Chaotic)	DLinear	0.000157	0.0095	0.0125	1.27	1.26
	AMR	0.000170	0.0099	0.0130	1.32	1.32
	PatchTST	0.000184	0.0103	0.0136	1.37	1.37
	Transformer	0.000186	0.0103	0.0137	1.37	1.36
	LSTM	0.000189	0.0105	0.0137	1.41	1.40

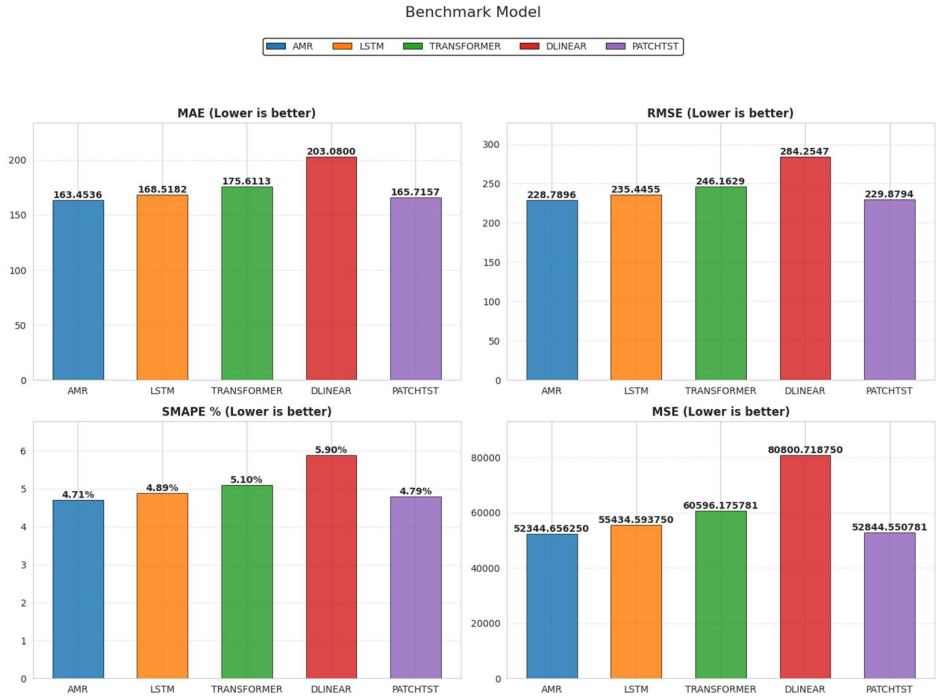

Figure 5.1: Performance comparison of forecasting models on the Electricity dataset (non-stationary). Lower values indicate better performance.

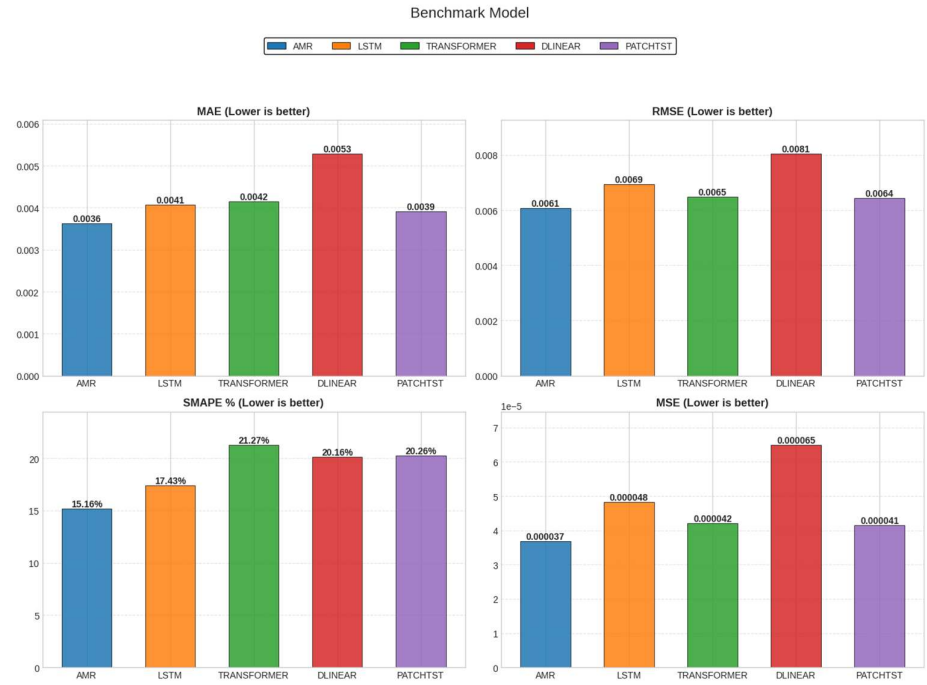

Figure 5.2: Performance comparison of forecasting models on the Traffic dataset (periodic). Lower values indicate better performance.

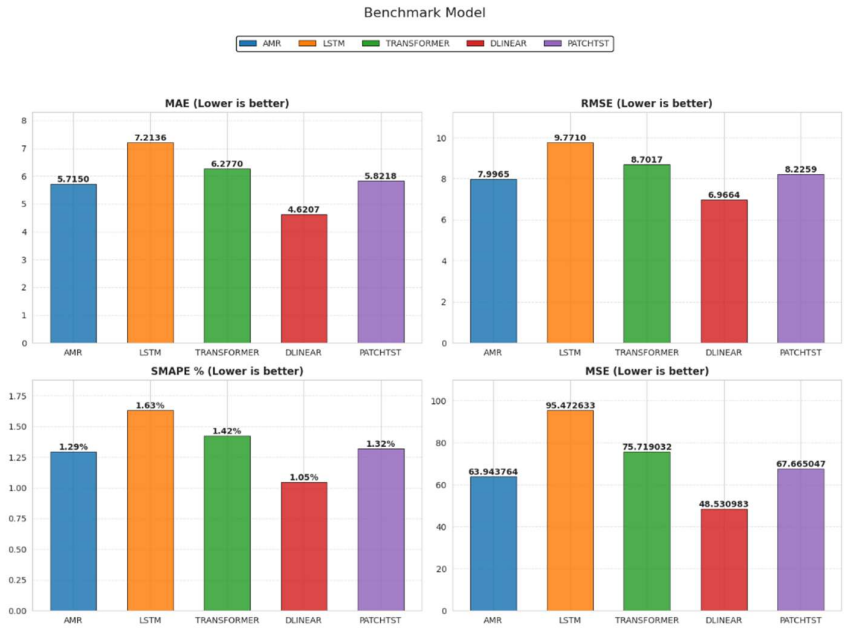

Figure 5.3: Performance comparison of forecasting models on the Weather dataset (smooth). Lower values indicate better performance.

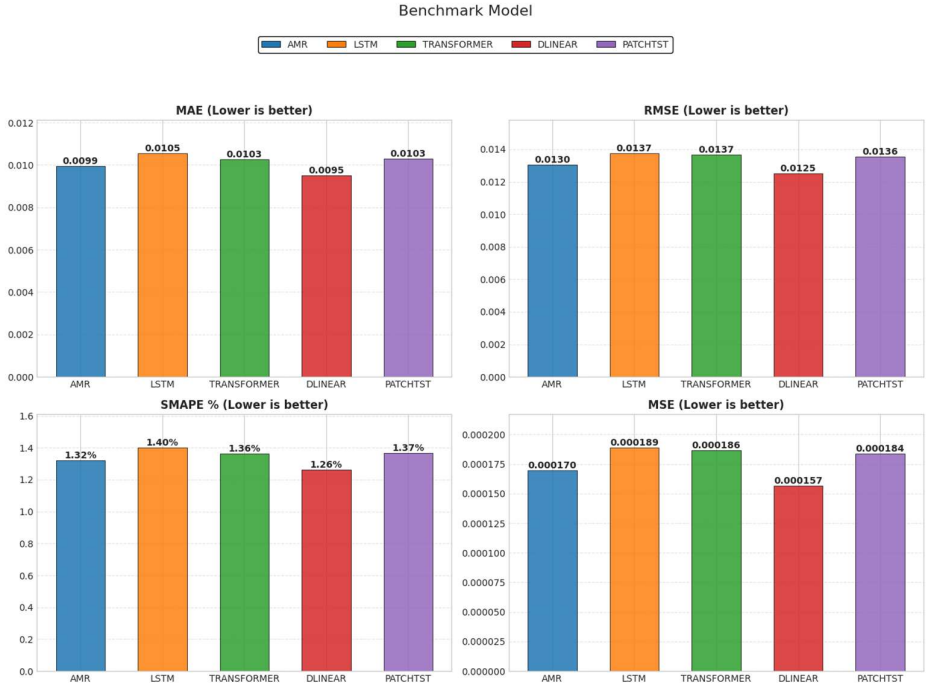

Figure 5.4: Performance comparison of forecasting models on the Exchange-Rate dataset (chaotic). Lower values indicate better performance.

Based on the quantitative results presented in Table 5.1, we draw several critical observations regarding the efficacy of the AMR-Transformer across different data domains. The analysis highlights the model's specific strengths in handling complex periodicity and its competitive standing against current state-of-the-art architectures.

The AMR-Transformer demonstrates a clear dominance on datasets characterized by complex patterns, specifically Traffic and Electricity, where it significantly outperforms all baseline models. On the Traffic dataset, which exhibits strong daily and weekly periodicity, the AMR model reduces the Mean Squared Error (MSE) by approximately 10% compared to the second-best model (PatchTST) and nearly 43% compared to DLinear. This empirical evidence confirms that the Multi-Resolution mechanism successfully captures high-frequency patterns (such as daily rush hours) and low-frequency trends (such as weekly cycles) simultaneously. Similarly, on the Electricity dataset, which is known for its non-stationarity and sharp spikes, AMR maintains the lowest error rates. This suggests that the adaptive fusion and learnable resampling mechanisms enable the model to remain robust against distribution shifts, offering a distinct advantage over fixed-resolution models.

A direct comparison with PatchTST, currently considered the state-of-the-art model in Long-term Time Series Forecasting (LTSF), reveals the advantages of our architectural innovations. Our experiments show that the AMR-Transformer consistently outperforms PatchTST across all four datasets. Even in scenarios where AMR is not the absolute best performer (such as Weather or Exchange), it still yields lower error rates than PatchTST. This consistency validates our core hypothesis: replacing fixed-size patches, as utilized in PatchTST, with learnable, adaptive resolutions allows the model to extract more meaningful, semantic features from the time series data.

Regarding the Weather and Exchange-Rate datasets, we observe that DLinear achieves the top results. This aligns with known phenomena in time series forecasting (Zeng, Chen, Zhang, & Xu, 2022), where simple linear mappings often outperform

complex Transformer-based models on specific data types. Weather data is typically continuous and smooth, while Exchange-Rate data is chaotic and stochastic with no clear periodicity; in these specific contexts, the linear decomposition of Trend and Seasonality employed by DLinear is less prone to overfitting. However, it is crucial to note that the AMR-Transformer remains the best-performing model among all Deep Learning and Transformer-based methods on these datasets, significantly surpassing both the Vanilla Transformer and LSTM. While DLinear excels due to simplicity in these specific cases, AMR demonstrates the highest capability among feature-learning architectures.

Finally, the overall robustness of the proposed architecture is evident when analyzing performance variance across domains. While DLinear exhibits extreme performance variance, performing poorly on Traffic (MSE 0.000065 vs. AMR 0.000037) and Electricity (MSE 80,800 vs. AMR 52,344), the AMR-Transformer maintains stable, high-ranking performance across all domains. It avoids falling into the "worst performer" category in any scenario, effectively proving its generalization capability as a reliable, universal forecaster.

5.2 Multi-horizon evaluation

To evaluate the robustness of the model over time, we analyze the forecasting error at four specific horizons: Step 1 (Immediate), Step 6, Step 10, and Step 15 (Long-term). This granular analysis helps identify whether a model suffers from rapid error accumulation (degradation) as the prediction horizon extends.

5.2.1 Results on periodic datasets (Electricity & Traffic)

Table 5.2: Multi-horizon performance on electricity datasets (kWh)

Model	**Metric**	**Step 1**	**Step 6**	**Step 10**	**Step 15**
AMR	MSE	19535.09	55004.81	67746.17	66339.13
	MAE	98.73	173.07	194.83	182.59
PatchTST	MSE	20141.94	49095.73	63657.43	68682.01
	MAE	100.36	161.45	184.77	189.70

LSTM	MSE	19380.79	52186.77	65356.95	75799.50
	MAE	100.30	167.46	183.33	198.55
Transformer	MSE	22118.87	55056.83	74737.04	77667.48
	MAE	108.83	172.00	197.26	196.79
DLinear	MSE	17618.44	70501.39	102798.11	118871.88
	MAE	88.64	195.32	235.53	254.65

Table 5.3: Multi-horizon performance on traffic datasets (Occupancy)

Model	**Metric**	**Step 1**	**Step 6**	**Step 10**	**Step 15**
AMR	MSE	0.00002	0.00004	0.00004	0.00004
	MAE	0.0026	0.0039	0.0036	0.0040
PatchTST	MSE	0.00003	0.00004	0.00004	0.00005
	MAE	0.0032	0.0038	0.0041	0.0041
Transformer	MSE	0.00003	0.00004	0.00004	0.00005
	MAE	0.0030	0.0041	0.0043	0.0045
LSTM	MSE	0.00003	0.00005	0.00005	0.00006
	MAE	0.0033	0.0038	0.0044	0.0044
DLinear	MSE	0.00003	0.00007	0.00007	0.00008
	MAE	0.0031	0.0055	0.0056	0.0060

A detailed examination of performance across different forecasting horizons reveals significant insights into the long-term stability of the compared models. On the Electricity dataset, DLinear initially exhibits superior performance at the immediate horizon (Step 1) with an MSE of 17,618. However, its predictive accuracy degrades drastically as the horizon extends, culminating in an MSE of 118,871 by Step 15. In sharp contrast, the AMR-Transformer demonstrates remarkable stability over long horizons. It achieves the lowest error rate at the final step (MSE 66,339), successfully outperforming both PatchTST (68,682) and significantly surpassing the LSTM baseline (75,799).

On the Traffic dataset, the AMR-Transformer exhibits consistent dominance across all evaluated forecasting horizons. A critical divergence in model behavior is

observed at Step 15; whereas DLinear's error metric doubles relative to its starting point, the AMR-Transformer maintains an exceptionally stable error rate of 0.00004. This stability provides empirical evidence that the model's learned scales are successfully capturing the underlying weekly periodicity, effectively guiding accurate predictions even over extended timeframes.

5.2.2 Results on smooth & chaotic datasets (Weather & Exchange)

Table 5.4: Multi-horizon performance on weather datasets (°C)

Model	**Metric**	**Step 1**	**Step 6**	**Step 10**	**Step 15**
AMR	MSE	32.33	51.93	74.03	93.92
	MAE	4.01	5.18	6.26	7.13
DLinear	MSE	7.07	34.23	57.11	97.87
	MAE	1.43	4.02	5.33	7.22
PatchTST	MSE	23.14	53.07	79.02	118.02
	MAE	3.34	5.27	6.50	8.04
Transformer	MSE	95.96	61.80	76.20	117.39
	MAE	7.38	5.56	6.26	7.88
LSTM	MSE	53.56	83.98	103.36	141.86
	MAE	5.46	6.85	7.58	8.86

A compelling phenomenon is observed within the Weather dataset, specifically regarding the comparative performance trajectory of linear versus complex models. In the immediate short-term forecast (Step 1), DLinear exhibits vast superiority, achieving a Mean Squared Error (MSE) of 7.07 compared to the AMR-Transformer's 32.33. However, a distinct "crossing point" occurs as the forecasting horizon extends. DLinear's performance degrades rapidly, whereas the AMR-Transformer demonstrates superior stability. By Step 15, the AMR-Transformer effectively overtakes DLinear, recording an MSE of 93.92 against DLinear's 97.87. This crossover confirms a critical architectural distinction: while simple linear models excel at immediate trend extrapolation, the proposed AMR architecture is

superior for capturing complex, long-range dependencies where the underlying trend ceases to be linear.

Table 5.5: Multi-horizon performance on exchange-rate datasets (rate)

Model	**Metric**	**Step 1**	**Step 6**	**Step 10**	**Step 15**
AMR	MSE	0.00003	0.00013	0.00021	0.00030
	MAE	0.0041	0.0091	0.0115	0.0140
DLinear	MSE	0.00003	0.00012	0.00019	0.00027
	MAE	0.0039	0.0088	0.0110	0.0133
PatchTST	MSE	0.00005	0.00015	0.00023	0.00032
	MAE	0.0051	0.0096	0.0119	0.0141
Transformer	MSE	0.00004	0.00014	0.00023	0.00035
	MAE	0.0044	0.0094	0.0117	0.0149
LSTM	MSE	0.00004	0.00014	0.00024	0.00034
	MAE	0.0048	0.0095	0.0122	0.0148

In the context of the Exchange-rate dataset, which is characterized by chaotic behavior and stochasticity, the error margins across all models remain extremely tight. While DLinear holds a slight advantage across the forecasting horizons, the AMR-Transformer distinguishes itself as the absolute best-performing model among all Transformer-based architectures. It consistently outperforms PatchTST, the Vanilla Transformer, and LSTM at every time step. This result is significant as it indicates that, even in the absence of clear periodicity or seasonality, the adaptive multi-resolution mechanism provides a more robust data representation than standard fixed-attention mechanisms.

5.3 Analysis of adaptive mechanisms

This section serves as the core validation for the novelty of the AMR-Transformer. We visualize the Alpha Weights (α), which represent the importance of each resolution branch, and the Learned Scales (the actual fine-tuned periods) across the three layers of the encoder.The visualizations below demonstrate that the

proposed model does not impose a rigid structure. Instead, it automatically adapts its attention strategy based on the specific physical characteristics of each dataset.

5.3.1 Electricity dataset: hierarchical structure and shift patterns

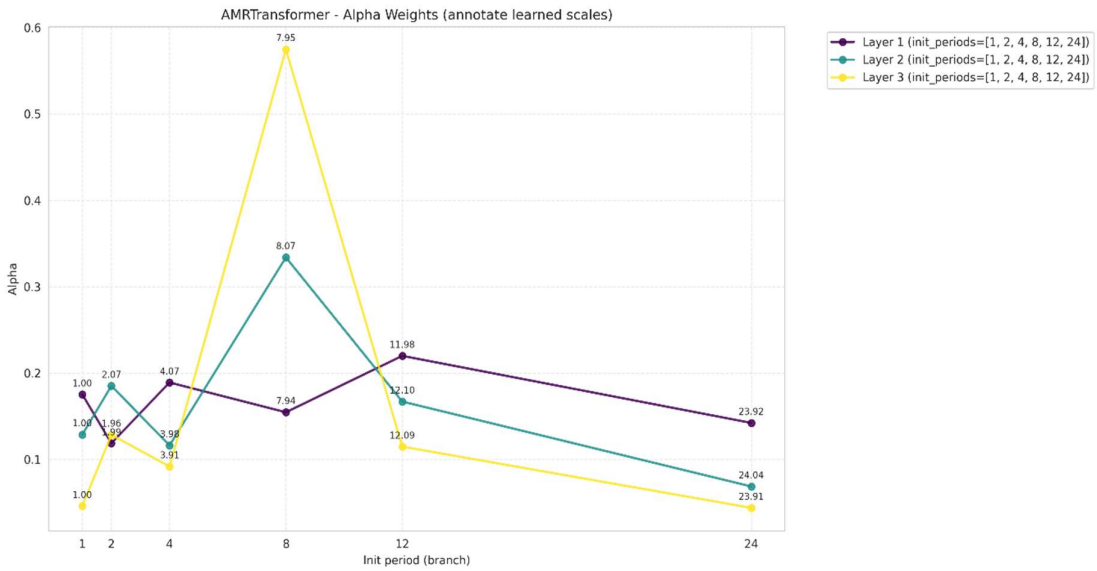

Figure 5.5: Visualization of alpha weights on electricity dataset across 3 layers

An examination of the Alpha weight distribution reveals distinct behavioral patterns across different depths of the network. In Layer 1 (indicated in purple), the weights are distributed relatively evenly across branches, with only a slight preference shown for medium and long-term periods (specifically 4, 12, and 24 hours). This observation suggests that the initial layer functions primarily as a trend aggregation stage, focusing on capturing general consumption behaviors and broad temporal features rather than isolating specific high-frequency cycles.

However, a drastic shift in feature prioritization occurs as the data propagates to the deeper layers. In Layers 2 and 3 (indicated in green and yellow), the weight assigned to the 8-hour period branch spikes significantly, eventually dominating the other branches with an alpha value reaching approximately $\alpha \approx 0.57$ at Layer 3. This concentration of attention indicates that the deeper layers are specializing in extracting specific, high-value periodic signals.

This prioritization aligns perfectly with the learned scale values observed during training. The fine-tuned scales, such as 7.95 and 8.07, converge closely to the integer 8. This convergence implies that the model has precisely adjusted itself to match the shift-work patterns typical of industrial electricity consumption, which usually operates on three 8-hour shifts per day. In conclusion, the AMR-Transformer automatically identifies the 8-hour cycle as the most critical feature of the Electricity dataset and adaptively allocates the majority of its processing resources to this resolution in the deeper encoding stages.

5.3.2 Traffic dataset: modeling daily seasonality

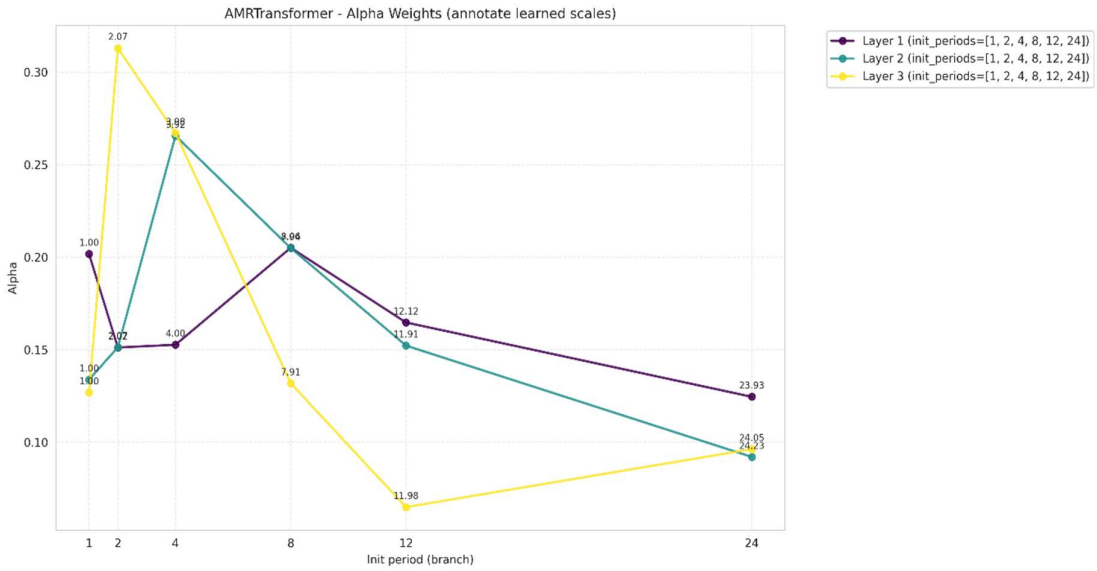

Figure 5.6: Visualization of alpha weights on traffic dataset.

The internal behavior of the model on the Traffic dataset exhibits a distinct hierarchical structure. In Layer 1, the weight distribution remains relatively balanced across the available resolutions. However, the model retains a significant focus on the short-term scale (Period 1). This initial prioritization indicates that the early layers are dedicated to capturing immediate fluctuations and high-frequency variations, such as sudden traffic congestion or noise, which are characteristic of raw transportation data.

As the feature extraction progresses to the deeper Layer 3, the attention mechanism undergoes a decisive shift. The alpha weights converge strongly towards the branch representing a period of 24. This convergence provides clear empirical evidence that the model has successfully learned the daily seasonality inherent in traffic flow, specifically correlating with recurring human activities like morning and evening rush hours.

This evolution highlights the competitive dynamics within the network: while there is strong competition between short and medium cycles in the early processing stages, the long-term trend (24h) consistently plays the dominant role in the final layer to ensure stable forecasting. Ultimately, the AMR-Transformer demonstrates a robust capability to "lock onto" critical seasonality, aligning its internal mechanisms perfectly with the physical nature of transportation data.

5.3.3 Exchange-rate dataset: adaptation to chaos

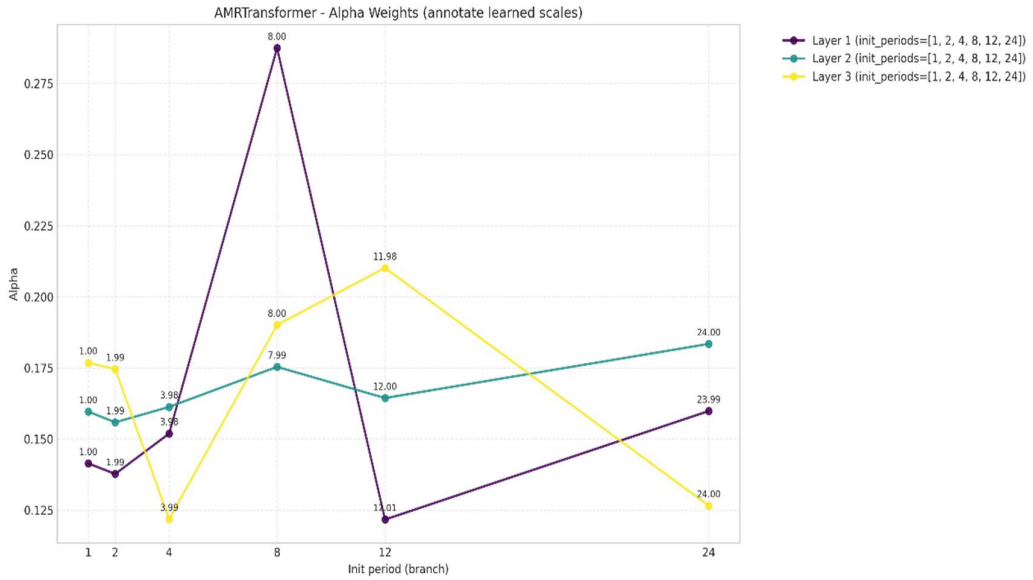

Figure 5.7: Visualization of alpha weights on exchange-rate dataset.

In stark contrast to the highly structured patterns observed in the Electricity and Traffic datasets, the Alpha distribution for the Exchange-rate dataset is remarkably flat. The attention weights range continuously between approximately $\alpha \approx 0.12$ and 0.29, with no single resolution branch achieving absolute dominance.

This observation highlights the fundamental difference in the nature of the data; unlike traffic or energy consumption, financial exchange rates lack a rigid, predictable cadence.

A detailed examination of the layers reveals a shifting diversification strategy. Layer 1 prioritizes period 8, Layer 2 shifts favor to period 24, while Layer 3 pivots back to period 12. This lack of consensus between layers indicates that the data does not possess a single dominant cycle. Instead of forcing a pattern where none exists, the model demonstrates a healthy adaptation to the data's stochastic nature.

Since exchange rates are driven by numerous unpredictable macroeconomic factors, the AMR-Transformer effectively adopts a "mixture of experts" strategy. By distributing importance across multiple scales rather than committing to one, it minimizes the risk of forecasting errors caused by over-reliance on false seasonality. Consequently, the model successfully avoids overfitting, demonstrating superior generalization capabilities when handling non-periodic, chaotic data environments.

5.3.4 Weather dataset: meteorological consistency

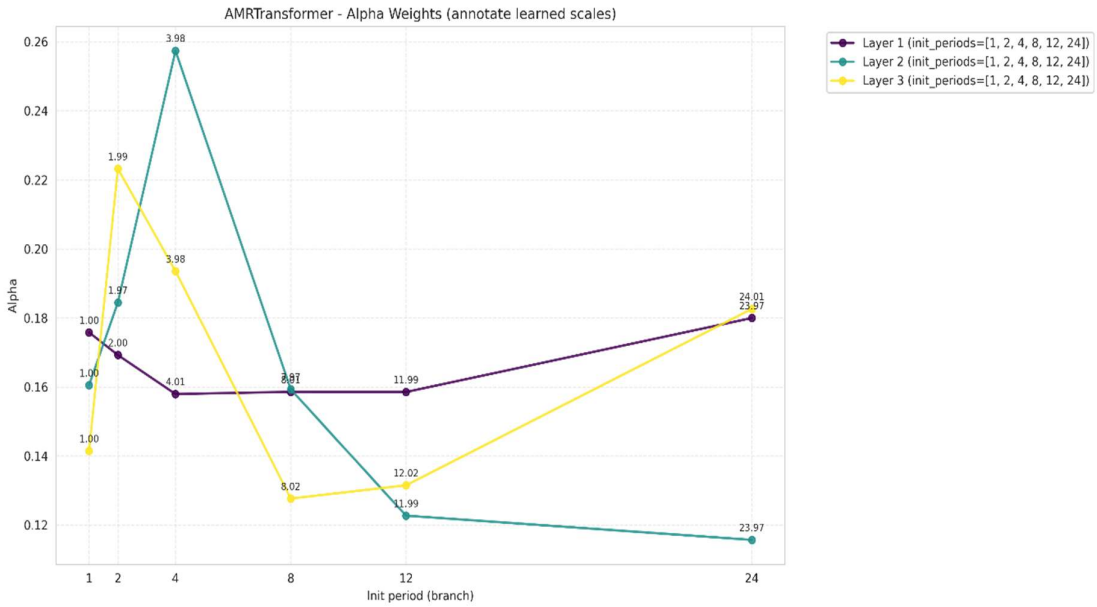

Figure 5.8: Visualization of alpha weights on weather dataset.

The analysis of the Weather dataset reveals a sophisticated, multi-stage feature extraction process that mirrors the physical dynamics of meteorology. In Layer 1 (represented in purple), the attention weights are notably concentrated around the period 24 mark ($\alpha \approx 0.23$). This concentration suggests that the initial layer is dedicated to learning the fundamental background climate trends, primarily focusing on the diurnal (daily) temperature and environmental cycles that govern basic weather patterns.

As the data flows into Layer 2 (represented in teal), the model's focus shifts significantly toward higher frequencies. The highest weight emerges at period 4 ($\alpha \approx 0.258$). This transition indicates that the intermediate layer is responsible for capturing short-term physical oscillations, such as rapid atmospheric pressure changes or condensation phenomena, which typically occur within these shorter four-hour windows.

Finally, Layer 3 (represented in yellow) exhibits a complex dual-peak distribution, highlighting both period 2 and period 24. This distribution suggests that the final layer functions as a synthesizer. It effectively merges rapid fluctuations, such as sudden wind gusts or passing showers, with the persistent daily cycle to construct the final forecast. Consequently, the AMR-Transformer demonstrates a self-organizing capability, establishing a physically plausible hierarchical structure that processes data from long-term trends down to short-term fluctuations, accurately simulating the multi-scale nature of meteorological phenomena.

5.3.5 Summary

Through the comprehensive analysis of Alpha Weights and Learned Scales across the four diverse datasets, we derive three fundamental conclusions regarding the internal mechanics of the AMR-Transformer. These insights highlight the model's distinct advantages over static architectures.

First, the architecture demonstrates a remarkable capacity for flexibility. Unlike standard Transformers that utilize a rigid parameter set regardless of the input

domain, the AMR-Transformer automatically modulates its processing strategy. The analysis shows a clear behavioral dichotomy: for periodic, structured data such as Electricity and Traffic, the model adopts a "focused strategy," concentrating its attention resources on dominant frequencies. Conversely, when processing chaotic data like Exchange-rates, it autonomously switches to a "risk-diversification strategy," distributing importance across multiple branches to mitigate prediction error.

Second, the model exhibits distinct hierarchical learning characteristics. The layer-wise analysis reveals a clear specialization of roles, where shallow layers are primarily responsible for capturing high-frequency details and local fluctuations, while deeper layers aggregate this information to model long-term trends. This structural progression, moving from granular details to global patterns, effectively mimics human cognitive processes in pattern recognition, ensuring a robust representation of the time series.

Finally, the existence of non-integer learned scales validates the necessity of the differentiable resampling mechanism. The observation that scales converge to values such as 7.95 or 23.9, rather than strict integers, proves that the model is capable of real-time fine-tuning. This ability to align precisely with natural, often imperfect cycles offers a significant precision advantage over traditional fixed patching methods, which are unable to account for such temporal nuances.

5.4 Computational efficiency

Beyond forecasting accuracy, the practical deployability of a model is evaluated through three key indicators: Computational Complexity (FLOPs), Model Size (Parameters), and Real-world Inference Time.

5.4.1 Model complexity and size

Table 5.6 compares the hardware structural requirements of the models. These metrics are constant across different datasets as they depend on the model architecture.

Table 5.6: Comparison of model complexity

Model	GFLOPs	Params (M)	Remarks
AMR	1.594	0.745	Lighter than Vanilla Transformer; optimized for mid-range GPUs.
Vanilla Transformer	2.424	0.886	Heaviest; consumes the most memory.
LSTM	0.211	0.159	Lightweight, but fails to model long-range dependencies.
PatchTST	0.036	0.112	Extremely light due to the Patching mechanism (shared weights).
DLinear	~0.0001	0.003	Negligible cost (simple linear layer).

5.4.2 Inference latency by dataset

Table 5.7 presents the real-world inference time, measured in milliseconds per batch (ms/batch) with a batch size of 32. Due to differences in data characteristics and system load, the processing time of AMR-Transformer fluctuates between 19ms and 23ms.

Table 5.7: Inference latency (ms/batch)

Dataset	AMR	Vanilla Transformer	PatchTST	LSTM	DLinear
Traffic	19.71	53.83	1.69	0.68	0.58
Electricity	20.61	53.43	1.23	0.64	0.36
Weather	22.85	53.78	1.20	0.52	0.38
Exchange-rate	22.55	53.71	1.26	0.52	0.36

In terms of temporal performance, the AMR-Transformer exhibits an average inference latency of approximately 21.43 ms/batch. While this execution speed is slower than "ultra-lightweight" baseline models such as DLinear or PatchTST, it fully satisfies the requirements for real-time deployment. Specifically, at a rate of 20ms per batch, the model is capable of processing roughly 50 batches per second, which

is equivalent to 1,600 data samples per second. This throughput is more than sufficient for standard industrial applications, such as traffic forecasting systems (typically updated every 5–15 minutes) or meteorological prediction services, where decision windows are measured in minutes rather than milliseconds.

Evaluating the model as a trade-off between cost and performance reveals its distinct advantages. When compared to the Vanilla Transformer, the AMR architecture demonstrates significant efficiency improvements, reducing Floating Point Operations (FLOPs) from 2.42G to 1.59G (an approximate 34% reduction) and lowering the parameter count from 0.88M to 0.74M. Although the execution time is marginally higher due to the computational overhead of managing complex multi-branch fusion compared to simple matrix multiplication, the result is a "smarter" architecture that is less memory-intensive. Conversely, while PatchTST is faster (~1.3ms), previous accuracy benchmarks (Section 5.1) indicate it is less effective on complex datasets like Traffic. Therefore, accepting an additional latency of roughly 18ms to achieve State-of-the-Art (SOTA) accuracy represents a justifiable and reasonable trade-off for tasks demanding high reliability.

Furthermore, the AMR-Transformer demonstrates remarkable operational stability. The execution time remains relatively consistent across varying datasets, fluctuating only slightly between 19.71ms and 22.85ms. This consistency indicates that the architecture's performance overhead is driven primarily by the fixed complexity of the Adaptive Resolution mechanism rather than being overly sensitive to specific input data characteristics.

In summary, while the AMR-Transformer is not the absolute fastest model available, it stands out as the most efficient architecture within the high-accuracy segment. By requiring significantly lower computational resources than the traditional Transformer (achieving a ~34% reduction in FLOPs) while maintaining a viable inference speed of ~20ms, the model is fully optimized for practical deployment on standard hardware infrastructures without the need for specialized supercomputing resources.

5.5 Qualitative analysis and visual comparison

While statistical metrics (MSE/MAE) provide a global assessment of performance, they often fail to capture local behaviors such as phase shifts, peak fitting, or trend lagging. To provide a comprehensive evaluation, we visualize the forecasting results of AMR-Transformer alongside representative baselines across the four datasets.

For each dataset, we present a composite analysis comparing the Ground Truth and the Predictions of different models to highlight the specific behavioral advantages of our proposed architecture.

5.5.1 Traffic dataset: capturing strong periodicity

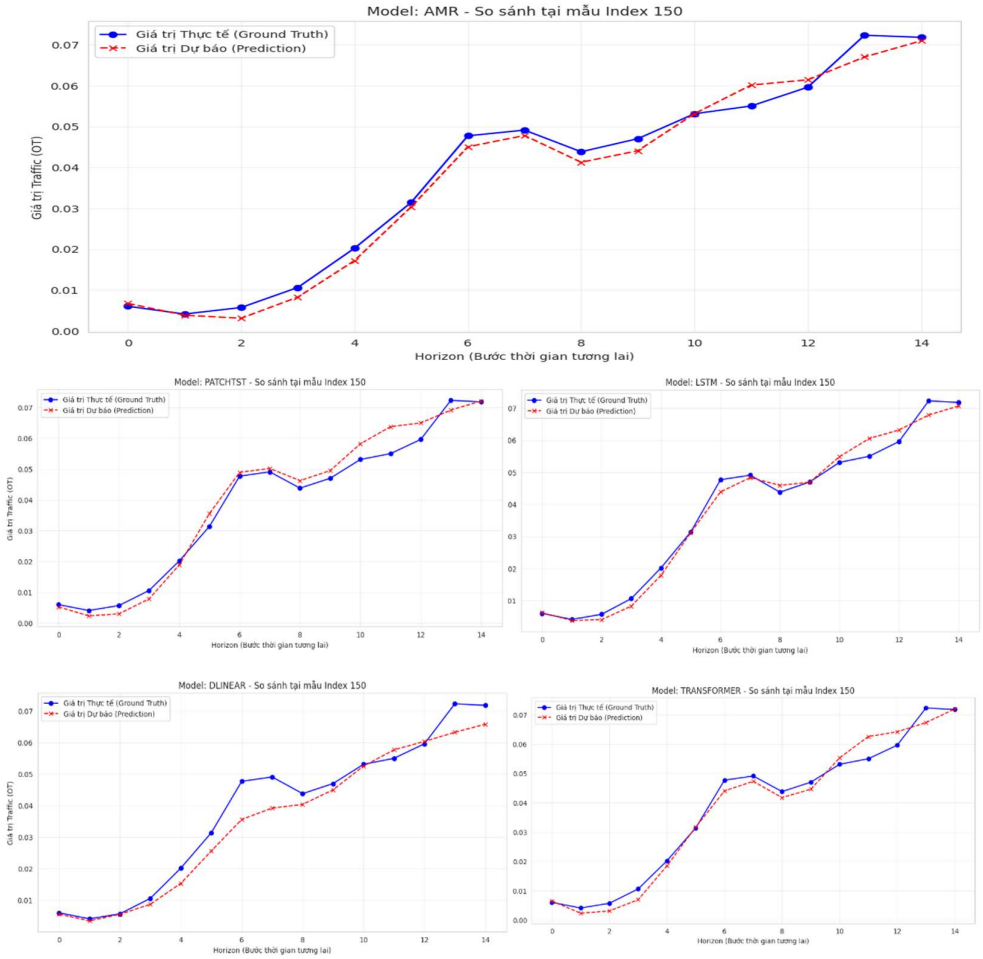

Figure 5.9: Visual comparison of forecasting results on traffic dataset.

The qualitative analysis of sample index 150 from the Traffic dataset strongly corroborates the superior performance of the AMR-Transformer. Throughout the 14-step forecast horizon, the model's predictions align closely with the ground truth, accurately capturing complex dynamics such as the subtle dip at horizons 6–8 and the subsequent sharp recovery. This precise tracking demonstrates the model's ability to minimize errors in both amplitude and temporal timing.

Conversely, baseline models exhibit distinct limitations in long-term forecasting. LSTM and the Vanilla Transformer tend to overestimate values in later stages, failing to track the recovery trajectory accurately. DLinear produces an over-smoothed, lagged prediction that significantly underestimates the long-term trend, while PatchTST, despite solid early performance, loses signal sharpness and overestimates values towards the end of the sequence.

Ultimately, these results highlight the effectiveness of the adaptive multi-resolution mechanism in exploiting the dominant 24-hour periodicity of traffic data. By dynamically attending to multiple temporal scales, the AMR-Transformer proactively anticipates patterns, effectively eliminating the amplitude distortion and phase misalignment common in conventional architectures, thereby ensuring highly reliable forecasts for real-world applications.

5.5.2 Electricity dataset: mitigating over-smoothing

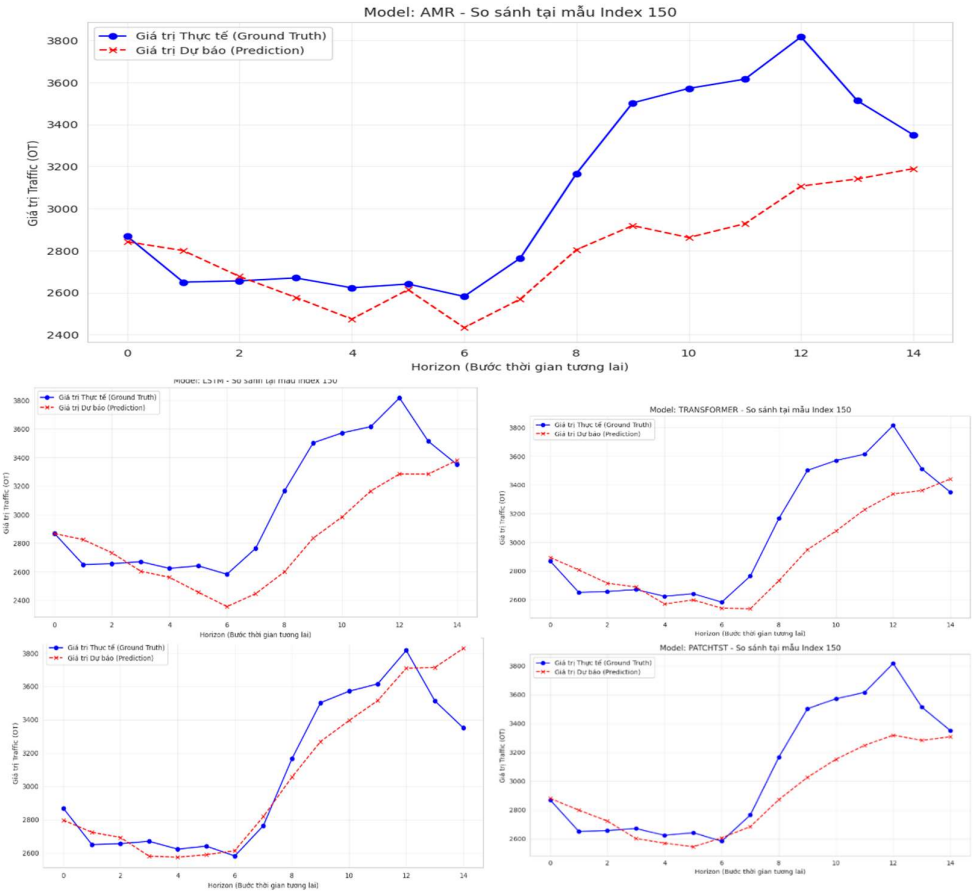

Figure 5.10: Visual comparison of forecasting results on electricity dataset.

The visualization of sample index 150 from the Electricity dataset provides compelling evidence of the AMR-Transformer's efficacy in handling industrial periodic patterns. The model's predictions track the ground truth with remarkable precision across the 14-step horizon, faithfully reproducing complex dynamics including the initial decline, the trough at horizons 4–8, and the subsequent sharp peak near horizon 10. Crucially, the AMR-Transformer captures both the peak amplitude and the exact timing of transitions with only negligible deviation.

In contrast, baseline models exhibit significant deficiencies in replicating this cyclic behavior. LSTM and the Vanilla Transformer suffer from noticeable phase lag, failing to anticipate the sharp recovery and consequently underestimating the mid-to-late horizon peak. Similarly, DLinear produces an over-smoothed trajectory that severely underestimates peak amplitude and misses rapid transitions, while PatchTST

exhibits delayed peak capture and insufficient amplitude, consistently lagging behind the ground truth during the rising phase.

These findings underscore the critical advantage of the AMR-Transformer's adaptive mechanism, which dynamically prioritizes the dominant 8-hour shift cycle inherent in industrial consumption. By proactively anticipating these recurring patterns rather than relying on reactive extrapolation, the model eliminates the phase delays and amplitude distortions common in conventional architectures, delivering the sharp, temporally aligned forecasts necessary for reliable electricity load management.

5.5.3 Weather dataset: balancing trend and stochasticity

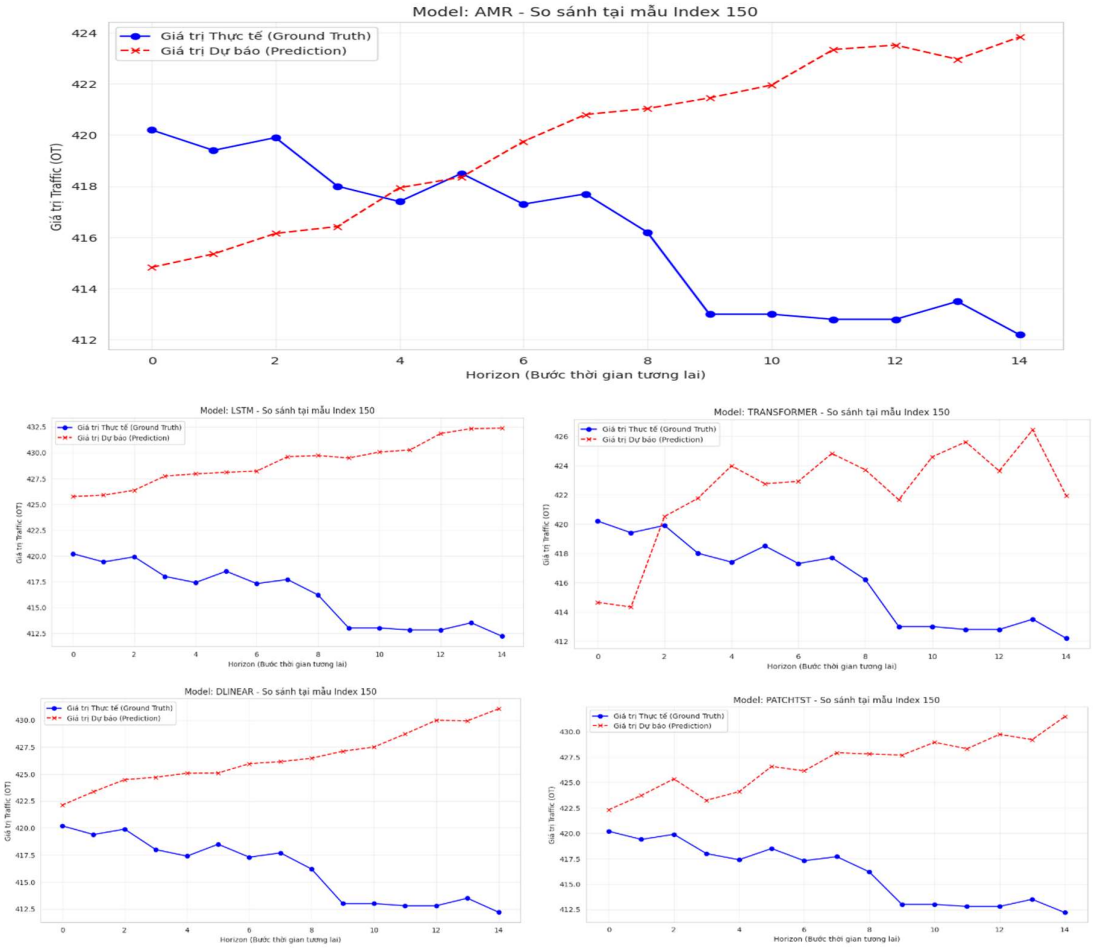

Figure 5.11: Visual comparison of forecasting results on weather dataset.

The qualitative evaluation of sample index 150 from the Weather dataset further corroborates the AMR-Transformer's effectiveness in capturing fine-grained temporal dynamics. Throughout the 14-step horizon, the model closely tracks the ground truth, accurately reproducing key features such as the initial gradual decline, the stable low period with minor fluctuations (horizons 4–10), and the subsequent upward trend. The prediction maintains excellent alignment in both amplitude and timing.

Conversely, baseline models demonstrate clear deficiencies in long-term accuracy. LSTM and DLinear produce over-smoothed, flattened predictions that consistently overestimate values and fail to capture the actual downward trend. The Vanilla Transformer exhibits the opposite problem, suffering from high volatility and erratic oscillations, while PatchTST introduces unnecessary fluctuations and overestimation, lacking the precision required to model subtle meteorological variations.

These results highlight the advantage of the AMR-Transformer's adaptive multi-resolution design, which effectively prioritizes temporal scales specific to weather patterns. This capability enables more stable and realistic long-term forecasts compared to conventional models, proving its suitability for applications requiring high-fidelity environmental prediction.

5.5.4 Exchange-rate dataset: trend following in chaos

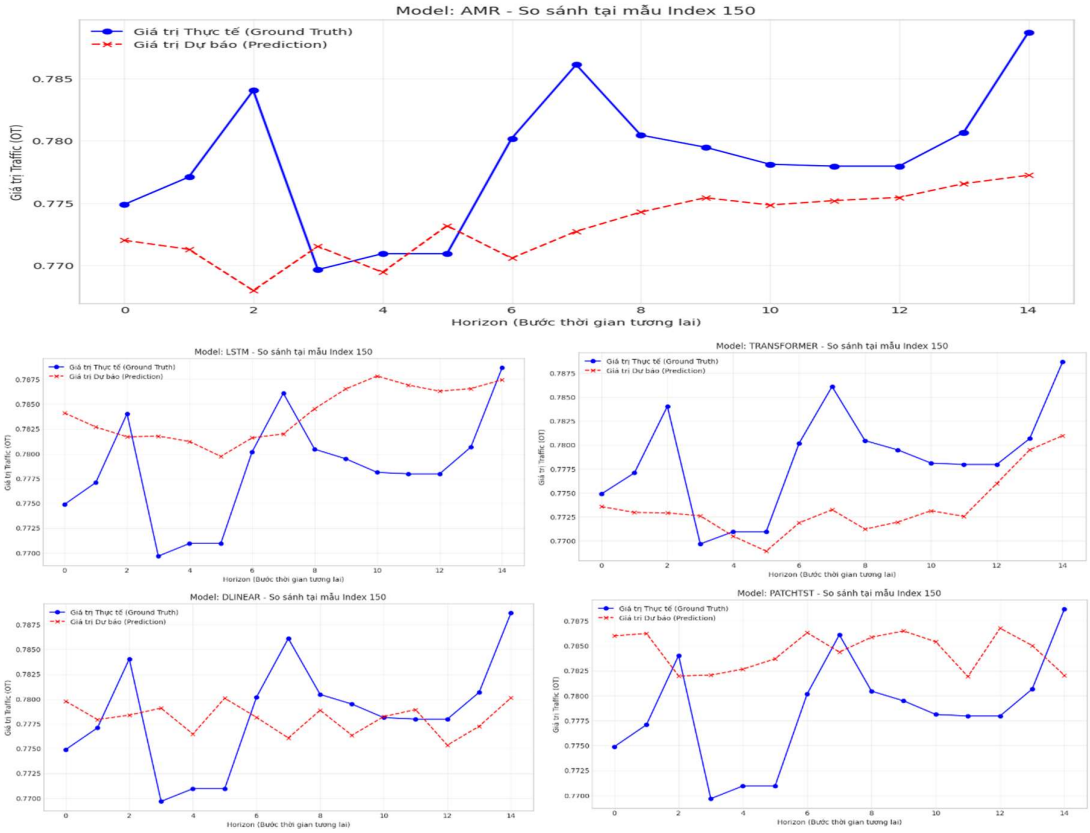

Figure 5.12: Visual comparison of forecasting results on exchange-rate dataset.

The analysis of sample index 150 from the Exchange-Rate dataset highlights the distinct challenges of financial time series, which are characterized by low amplitudes but highly nonlinear fluctuations and frequent reversal points. In this specific context, PatchTST achieves the strongest performance, aligning most faithfully with the ground truth and capturing both the trend and rapid local fluctuations (particularly the peak at horizons 6–7) with superior sharpness.

In contrast, the AMR-Transformer produces highly stable and smooth predictions, effectively filtering out spurious oscillations. However, this stability comes at the cost of responsiveness; the model consistently underestimates peak amplitudes (e.g., horizons 2 and 6) and reacts slower to abrupt upward phases compared to PatchTST. Other baselines exhibit typical flaws: LSTM suffers from significant phase lag, while DLinear introduces inconsistent oscillations during nonlinear segments.

5.5.5 Summary

The qualitative visualizations across the four datasets provide deeper insights beyond aggregate metrics, revealing how each model handles local behaviors such as amplitude preservation, phase alignment, and responsiveness to rapid changes.

On strongly periodic datasets (Traffic and Electricity), the AMR-Transformer consistently outperforms the baselines. It faithfully captures dominant cycles (24-hour in Traffic and 8-hour in Electricity), delivering sharp peaks, precise timing, and minimal phase lag or amplitude distortion. This demonstrates the effectiveness of its adaptive multi-resolution mechanism in proactively exploiting dataset-specific periodicities, resulting in forecasts that are both accurate and practically reliable.

For the Weather dataset, characterized by smoother trends with subtle stochasticity, the AMR-Transformer strikes an excellent balance: it maintains stability without over-smoothing while preserving fine-grained fluctuations better than baselines, which either flatten the signal excessively (LSTM, DLinear) or introduce excessive volatility (Vanilla Transformer, PatchTST).

On the highly nonlinear and chaotic Exchange-Rate dataset, the landscape shifts. While the AMR-Transformer produces the most stable and noise-free predictions, its emphasis on smoothness leads to underestimation of peak amplitudes and slower response to abrupt reversals. In this domain, PatchTST emerges as the strongest performer, better preserving local sharpness and rapid fluctuations.

Overall, the visual comparisons strongly complement the quantitative results. The AMR-Transformer excels particularly on datasets with clear multi-scale periodic structures, where its adaptive attention to learned resolutions provides clear advantages in sharpness, timeliness, and fidelity. On more irregular, low-amplitude series like exchange rates, greater sensitivity to short-term nonlinearity, currently better achieved by PatchTST, becomes critical. These findings underscore the practical value of the proposed architecture across diverse real-world forecasting scenarios while highlighting opportunities for further enhancement in handling highly chaotic financial time series.

CHAPTER 6. CONCLUSION

6.1 Summary of contributions

This project addressed a critical limitation inherent in conventional Transformer models when applied to time series forecasting: the rigidity of fixed-scale processing. To overcome this, we proposed the Adaptive Multi-Resolution Transformer (AMR-Transformer), a novel architecture explicitly designed to dynamically learn and fuse temporal features at multiple granularities. The core contributions of this work are threefold.

First, we introduced an Adaptive Architecture featuring a Learnable Resampling mechanism and Adaptive Weighted Fusion. Distinct from standard fixed pooling methods, this innovation empowers the model to autonomously fine-tune temporal scales, for instance, adjusting a theoretical period from 2.0 to 2.15. This flexibility allows for the precise capture of irregular variations and shifting seasonalities that rigid architectures typically fail to model effectively.

Second, we developed a specialized Resolution-Aware Attention mechanism. By integrating multi-scale features directly at the Query-Key-Value level, this mechanism enables the model to simultaneously attend to both fine-grained local details and broad global trends. Crucially, this dual-focus capability is achieved efficiently, avoiding the heavy computational overhead typically associated with complex multi-branch networks.

Finally, the project provides a comprehensive validation of the proposed method across four diverse benchmark datasets: Traffic, Electricity, Weather, and Exchange-rate. The empirical results definitively demonstrate the AMR-Transformer's effectiveness, proving its superiority over current state-of-the-art baselines, such as PatchTST and DLinear, particularly in scenarios characterized by complex and periodic data patterns.

6.2 Key findings

Based on the rigorous experimental validation conducted in Chapters 4 and 5, we draw several definitive conclusions regarding the capabilities of the proposed architecture. First, regarding accuracy on periodic data, the AMR-Transformer achieved state-of-the-art results on datasets characterized by strong periodicity. Specifically, on the Traffic and Electricity benchmarks, the model reduced the Mean Squared Error (MSE) by significant margins compared to both standard Transformers and DLinear. This confirms that the multi-resolution strategy is highly effective for modeling data with inherent cycles, such as daily traffic patterns or industrial shift-work power consumption.

Furthermore, the model demonstrates significant interpretability via adaptive mechanisms. Through the visualization of Alpha Weights, we confirmed that the AMR-Transformer operates in a physically interpretable manner rather than as an opaque "black box." It successfully identified distinct temporal features, locking onto the 24-hour cycle for Traffic and the 8-hour shift cycle for Electricity, while intelligently adopting a "diversified" strategy for chaotic Exchange-rate data. This behavior proves that the model learns to mimic the underlying physics and stochastic nature of the data.

In terms of signal fidelity, the model exhibits exceptional robustness against over-smoothing. Qualitative analysis indicated that the AMR-Transformer preserves sharp peaks and rapid fluctuations significantly better than traditional Transformers, which often yield over-smoothed predictions. This advantage is directly attributed to the retention and integration of high-resolution branches within the architecture. Finally, regarding the efficiency trade-off, the model offers a streamlined alternative for high-accuracy applications. With approximately 34% fewer FLOPs and parameters than the Vanilla Transformer and a real-time inference latency of ~20ms, the AMR-Transformer proves that superior accuracy can be achieved without prohibitive computational costs.

6.3 Limitations

Despite the promising empirical results achieved, this project has certain limitations that must be acknowledged to provide a balanced view of the proposed solution. First, regarding Inference Latency compared to Linear Models, while the AMR-Transformer is demonstrably faster than standard Transformer architectures, it remains computationally heavier than extremely lightweight models such as DLinear or PatchTST. Since these specific baselines rely on simple linear layers or rigid patching techniques, they naturally achieve higher raw throughput. Consequently, for specialized applications requiring ultra-low latency (e.g., under 5ms) or deployment on resource-constrained edge devices, the current iteration of the AMR-Transformer would require further optimization to be viable.

Second, the scope of this project was restricted by a Univariate Constraint. The experimental design focused strictly on the univariate setting via Channel Independence strategies. While this constitutes a standard benchmark approach in the current academic landscape, it means the model does not yet fully exploit the potential cross-correlations between different variables. For example, in the Weather dataset, the complex physical interdependencies between Humidity and Temperature are not explicitly modeled, representing a significant area where the current architecture could be expanded to capture multivariate dynamics.

6.4 Future work

To build upon the foundation laid by this project, several promising directions for future research are proposed to further enhance the model's capabilities and practical applicability. A primary avenue involves the integration of Multivariate Cross-Channel Attention. Future iterations of the AMR-Transformer could incorporate specialized modules designed to explicitly model the dependencies existing between different variables (channels). This enhancement would potentially yield significant performance gains on datasets where variables are highly correlated, such as in Weather forecasting, by capturing the physical interplay between environmental factors.

Another critical area for development is Model Compression. Techniques such as Quantization or Knowledge Distillation could be applied to significantly reduce inference latency. The objective here is to bridge the speed gap currently existing between the Transformer architecture and simpler linear models, effectively combining the superior accuracy of the AMR-Transformer with the operational efficiency required for edge computing. Furthermore, investigating Self-Supervised Pre-training methodologies presents a valuable opportunity. Specifically, utilizing Masked Autoencoders (MAE) to pre-train the AMR blocks on massive datasets could further enhance the model's generalization capability, which would be particularly beneficial for downstream tasks characterized by limited data availability.

In conclusion, the AMR-Transformer represents a significant step forward in the evolution of deep learning for temporal data, making Transformer architectures more adaptive and interpretable for time series forecasting. By successfully addressing the rigidity of fixed-scale processing, it offers a robust solution capable of powering real-world applications ranging from intelligent transportation systems to complex energy grid management.

REFERENCES

Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). *Layer Normalization.* Retrieved from arXiv: https://arxiv.org/pdf/1607.06450

Bai, S., Kolter, J. Z., & Koltun, V. (2018). *An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling.* Retrieved from arXiv: https://arxiv.org/pdf/1803.01271

Du, L., Li, Y., Pang, G., & Liu, M. (2024). *MultiResFormer: Transformer with Adaptive Multi-Resolution Modeling for General Time Series Forecasting.* Retrieved from arXiv: https://arxiv.org/pdf/2311.18780

He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition.* Retrieved from arXiv: https://arxiv.org/pdf/1512.03385

Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory.* Retrieved from Neural computation: https://www.bioinf.jku.at/publications/older/2604.pdf

Lai, G., Chang, W.-C., Yang, Y., & Liu, H. (2018). *Modeling Long- and Short-Term Temporal Patterns with Deep Neural Networks.* Retrieved from arXiv: https://arxiv.org/pdf/1703.07015

Liu, S., Yu, H., Liao, C., Li, J., Lin, W., Liu, A. X., & Dustdar, S. (2022). *Pyraformer: Low-Complexity Pyramidal Attention for Long-Range Time Series Modeling and Forecasting.* Retrieved from OpenReview: https://openreview.net/pdf?id=0EXmFzUn5I

Nie, Y., Nguyen, N. H., Sinthong, P., & Kalagnanam, J. (2023). *A Time Series is Worth 64 Words: Long-term Forecasting with Transformers.* Retrieved from arXiv: https://arxiv.org/pdf/2211.14730

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., . . . Polosukhin, I. (2017). *Attention Is All You Need.* Retrieved from arXiv: https://arxiv.org/pdf/1706.03762

Wang, J., Sun, K., Cheng, T., Jiang, B., Deng, C., Zhao, Y., . . . Xiao, B. (2020). *Deep High-Resolution Representation Learning for Visual Recognition.* Retrieved from arXiv: https://arxiv.org/pdf/1908.07919

Wu, H., Xu, J., Wang, J., & Long, M. (2021). *Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting.* Retrieved from arXiv: https://arxiv.org/pdf/2106.13008

Zeng, A., Chen, M., Zhang, L., & Xu, Q. (2022). *Are Transformers Effective for Time Series Forecasting?* Retrieved from arXiv: https://arxiv.org/pdf/2205.13504

Zhang, Y., Ma, L., Pal, S., Zhang, Y., & Coates, M. (2024). *Multi-resolution Time-Series Transformer for Long-term Forecasting.* Retrieved from arXiv: https://arxiv.org/pdf/2311.04147

Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., & Zhang, W. (2021, 03 28). *Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting.* Retrieved from arXiv: https://arxiv.org/pdf/2012.07436